

Agentic模型总体趋势

Agent定义：能用digital工具完成现实任务的智能体

OpenAI: reasoning, 工具和长上下文

Agents = Reasoning + Tools + Long Context
Tools: browsing / search, computer use, canvas, etc.

未来是innovator=agent+创造力(需人机协作)

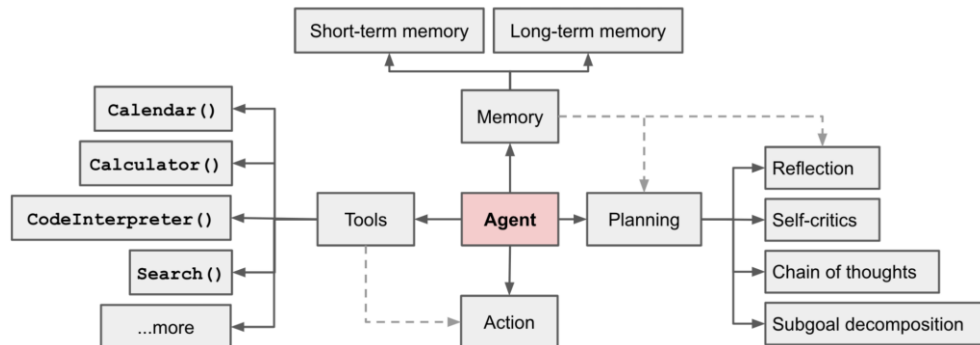
Co-Innovators = Agents + Creativity (human-AI collaboration)

Anthropic判断使用agent的条件：复杂+高价值+可完成+低错误成本

Checklist: "Should I build an agent"

Is the task complex enough?	No → Workflows Yes → Agents
Is the task valuable enough?	<\$0.1 → Workflows >\$1 → Agents
Are all parts of the task doable?	No → Reduce scope Yes → Agents
What is the cost of error/error discovery?	High → Read-only/human-in-the-loop Low → Agents

Agent=行动+规划+记忆+工具使用



Agent体验

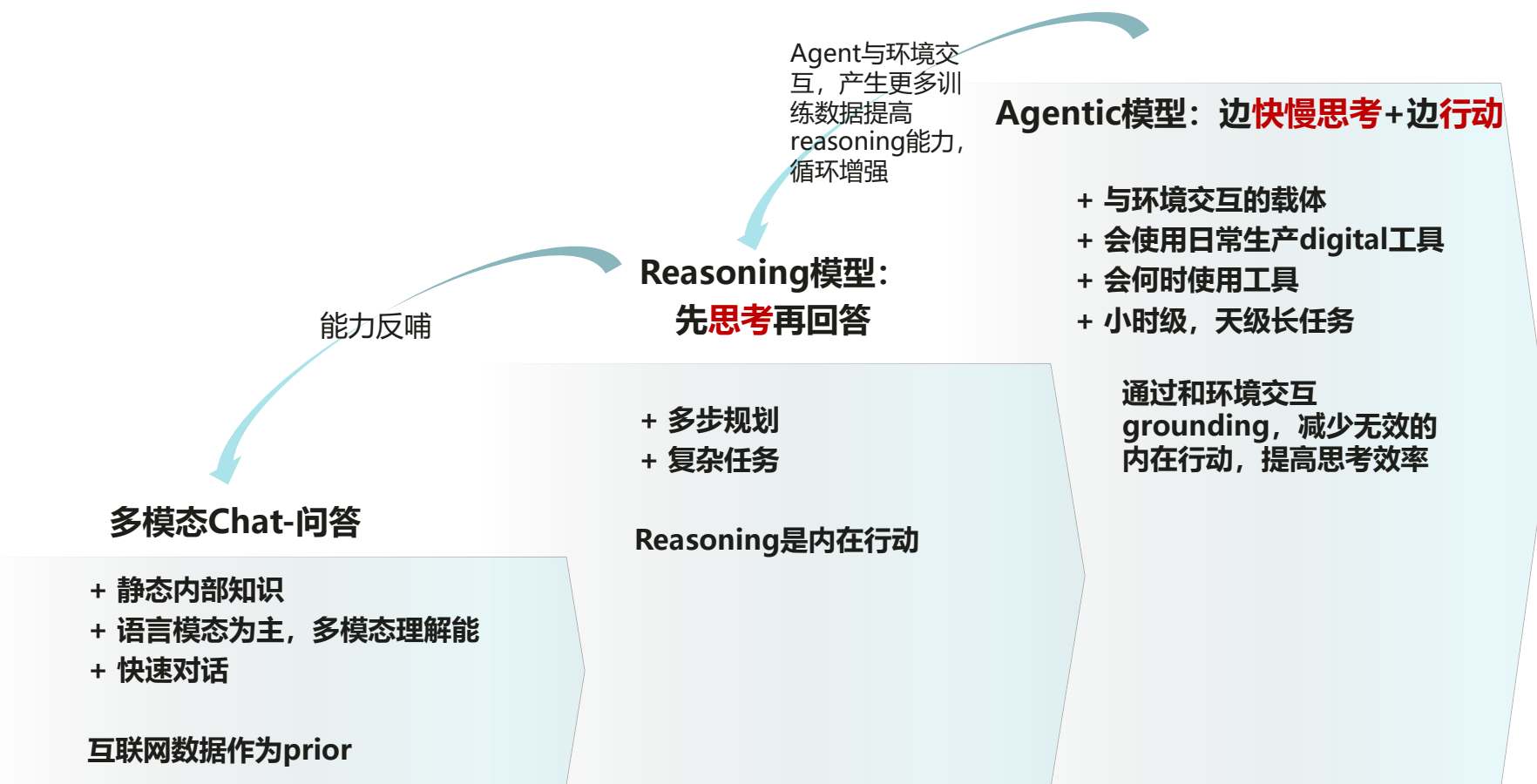
任务复杂度~耗时

完成效率~价值\$-成本\$

完成准确性~pass@k;

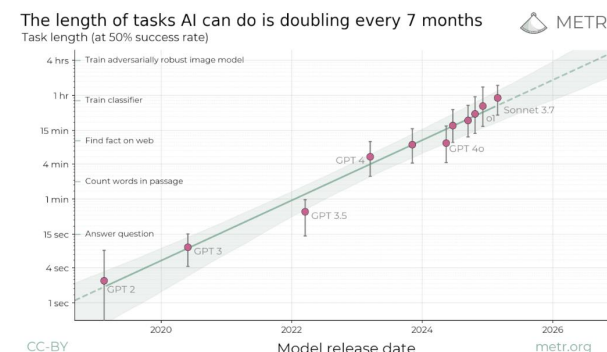
完成鲁棒性~pass^k【缺乏】

Why Agent?: 从问答到行动的转变, 利用现实工具真正解决现实问题; agent是未来下一跳的基础也反哺reasoning

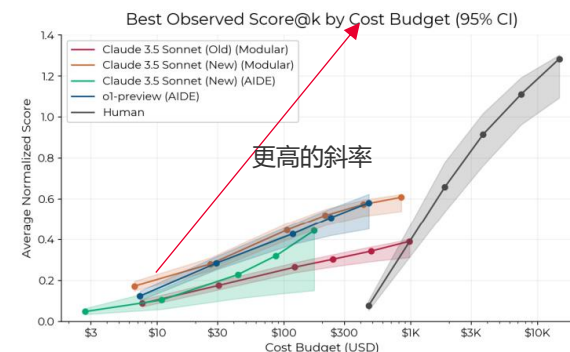


解决现实问题

用Agent替换/增强现实的digital生产工具, 直接产生经济价值



未来能可靠地完成长任务



未来要比人的边际产出效率更高

Agentic能力迭代加速，已具备特定易验证工具的能力；agentic模型在25年H2爆发，具备商业价值；头部基于自身优势，比拼全栈整合能力和迭代速度，26H1将见重量级应用

标志性能力	Reasoning模型	Tool use模型	原生agentic模型-长任务，调用所有工具		
趋势1-任务长度	(pass@1>50%, pass^k~人类)				
	0-5mins	5-45mins	>1h	~24h	1-7天
趋势2-调用工具数	Coding为主	<10个，以搜索/代码/桌面为主	100个的数量级，能使用更多生产工具	泛化到所有工具， 自优化工具	创造新工具
趋势3-记忆长度	Stateless	Stateful; 上下文1M	单体Life-long记忆; 上下文10M	群体记忆	
OpenAI	o3	Deep Research Operator Codex	GPT-5	GPT-N	
Anthropic	Claude 3.7	Computer use 谷歌workspace	Claude 4		
Deepmind	Gemini 2.5 Deep Think	Deep ReSearch Astra Mariner Jules: Gemini Code Assist	Gemini 3	Gemini 4	AI Co-scientist
				AlphaEvolve 2	
字节	Seed V1.5	Seed-ReTool Seed-Coder	Seed v2+抖音功能		
腾讯	混元T1		混元+微信小程序		
	25Q1	25Q2	NOW	26H2	2027

25模型迭代加速，RL开启数据飞轮，从6~12个月到3~6个月。预计H2，通过端到端训练产生第一批原生agentic模型，具备多模态思考和自主工具调用能力，领域能力进一步增强；26H1更多的现实环境中的digital tool将能被模型自主调用；多模态+thinking+coding三个能力将compound实现能力阶跃；结合用户历史数据，将出现个性化的全天候全能助手，实时响应通用任务。

- 模型优势（OpenAI, A）：借助模型优势，25H2推出各类agent，coding能力增强；25年底有集成各种工具能力的单一模型/产品；在26年继续丰富可交互的环境，使agentic模型能力泛化
- 互联网优势（腾讯，meta）：agentic RL scaling需要和真实环境快速交互，将会有reward scaling可扩展的reward系统；互联网是很好的交互环境，能发挥超级APP的优势
- 结合优势（谷歌，字节）：既有强大的模型，又有互联网环境优势，将结合自身优势产品打造agent形态，通过和用户的数据飞轮加速RL，实现模型-产品加速迭代；预计26H2前可见雏形

26H2: Agentic模型后期的能力将可以完成现实真实任务，对实际工作加速，因此拥有更先进agentic模型的公司将有更强大，更快的迭代能力，从而更快地开发下一代模型，算力infra和产品：build AI to accelerate AI Infra for AI; 将看到在自演进agent的军备竞赛，落后的团队可能将越发吃力

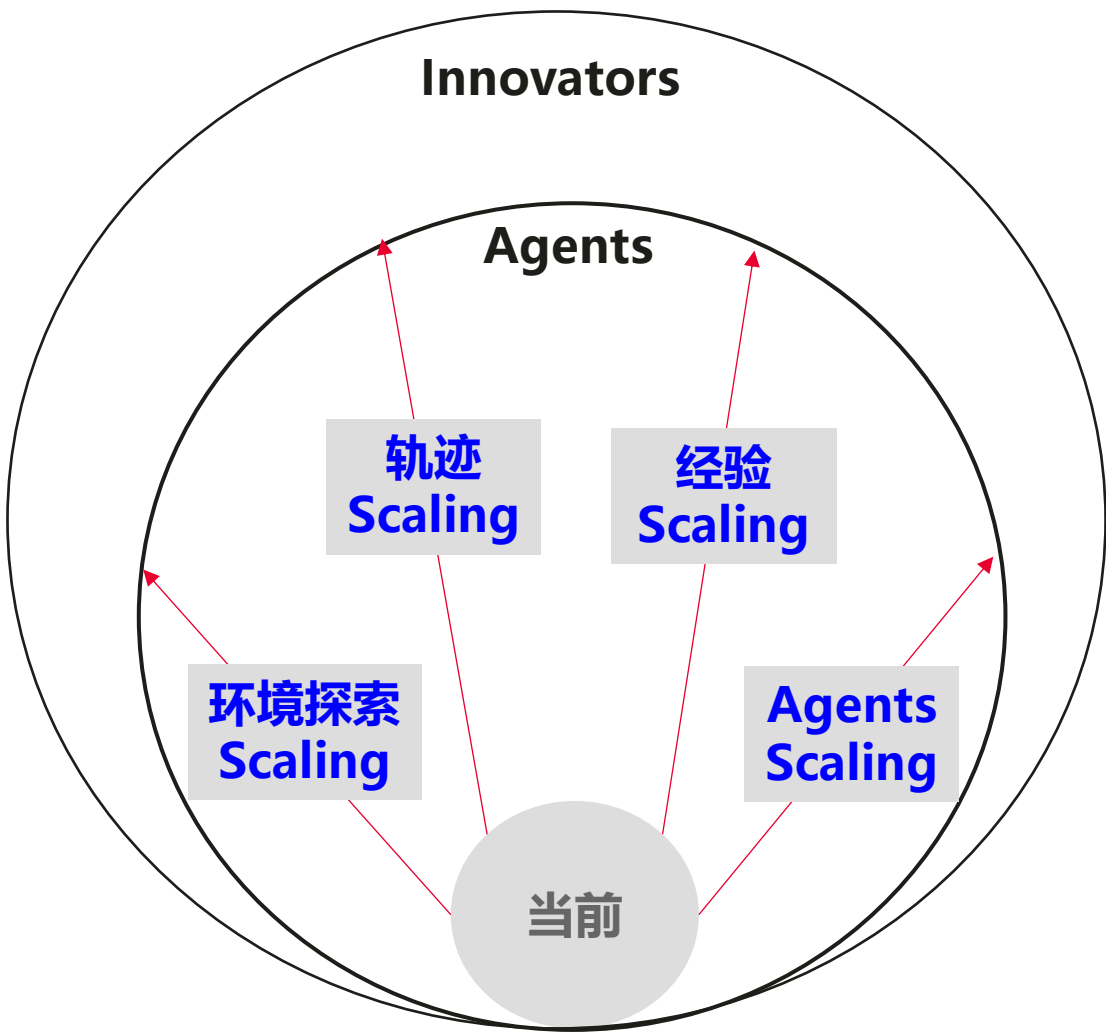
AI Infra: 随着RL负载复杂度提升，多程序+多阶段+多模型+多模态+多工具的特点，RL框架要支持异步RL和高效系统调度资源，满足agentic RL训练，提高端到端吞吐；业界正尝试从veRL等框架扩展，如skyRL等；RL框架之间的差距可达100-1000倍，效果越好的框架将支持模型的快速迭代

Agentic能力迭代加速，已具备特定易验证工具的能力；agentic模型在25年H2爆发，具备商业价值；头部基于自身优势，比拼全栈整合能力和迭代速度，26H1将见重量级应用

标志性能力	Reasoning模型	Tool use模型	原生agentic模型-长任务，调用所有工具	自优化agent	自演进agent	
趋势1-任务长度 (pass@1>50%, pass^k~人类)	0-5mins	5-45mins	>1h	~24h	1-7天	
趋势2-调用工具数	Coding为主	<10个，以搜索/代码/桌面为主	100个的数量级，能使用更多生产工具	泛化到所有工具， 自优化工具	创造新工具	
趋势3-记忆长度	Stateless	Stateful；上下文1M	单体Life-long记忆；上下文10M	群体记忆		
OpenAI	o3	Deep Research Operator Codex	GPT-5	GPT-N		
Anthropic	Claude 3.7	Computer use 谷歌workspace	Claude 4			
Deepmind	Gemini 2.5 Deep Think	Deep ReSearch Astra Mariner Jules: Gemini Code Assist	Gemini 3	Gemini 4 AlphaEvolve 2	AI Co-scientist	
字节	Seed V1.5	Seed-ReTool Seed-Coder	Seed v2+抖音功能			
	25Q1	25Q2	NOW	25H2-26H1	26H2	2027

系统	+ 异步agentic RL训推				The future of the Gemini models	
基模					- Omnimodal (r) Already have image + audio generative, next is video - Early experiments on Diffusion (r) - Agentic by default (m) Research class tool calling and tool use, but more than that, models are becoming agents - Reasoning scaling will continue (s) Research breakthrough after research breakthrough - More small models (s) More to share here soon! - Infinite context (r) The way attention and context works today, this is never going to be possible. We need new innovation at the core architecture level to enable this	
RL算法	+ RLVR				+ 原生视频生成	
reward	+ rule based+GRM		+ reward scaling		+ 自主在线学习的新算法？	
环境			+ intrinsic reward		+ 互联网，软件，app	
长序列/记忆	+ web, 编译器				+ 世界模型	
					+ 无限上下文	

Agentic AI演进趋势：新Scaling方式牵引Agentic模型能力快速进步

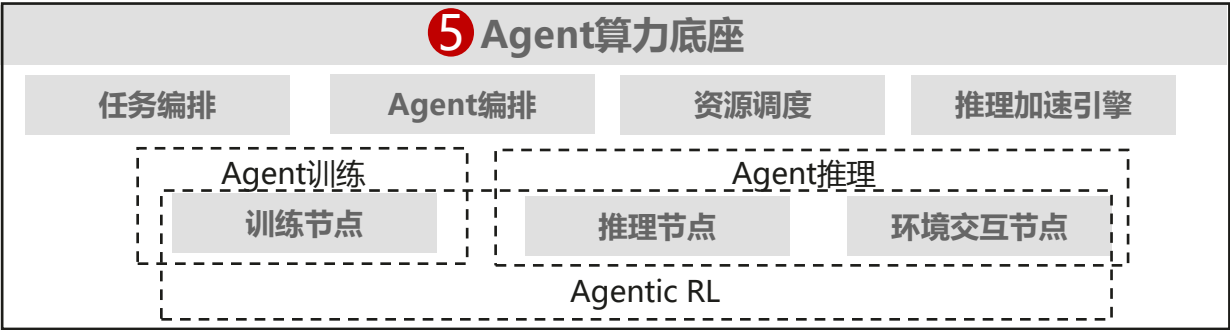
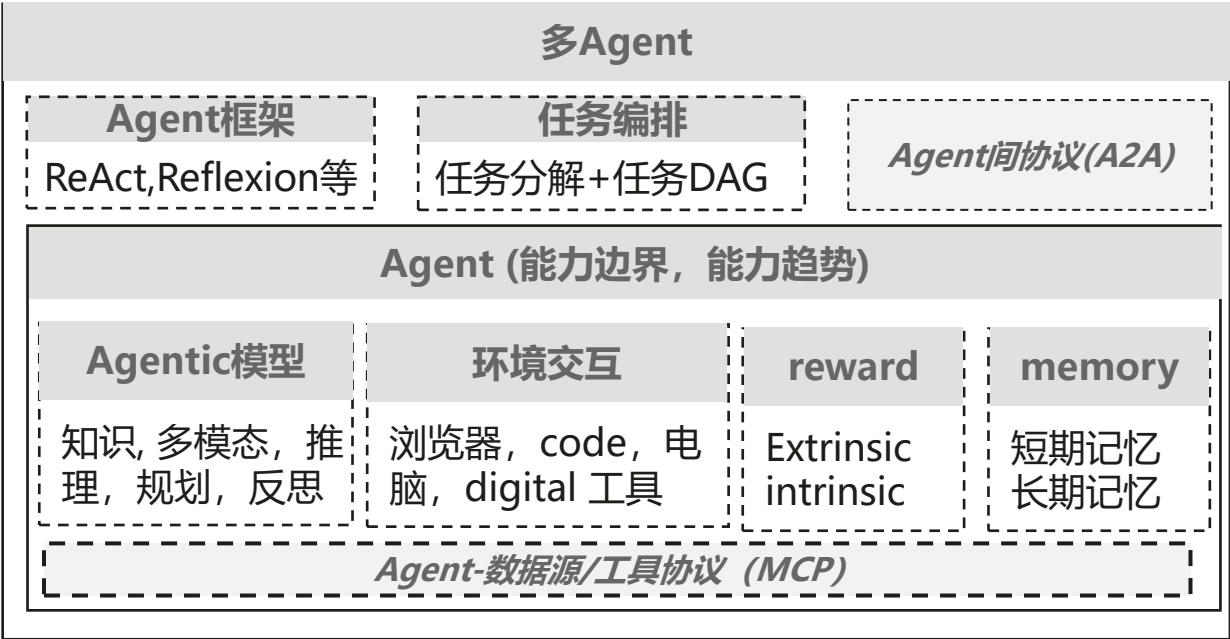


Agentic AI 仍在早期阶段，环境探索Scaling、轨迹Scaling、经验Scaling和Agents Scaling将极大提升其能力。

核心趋势

- ①**开放环境探索**：从单环境探索到复杂环境联合探索，开放奖励成为关键，世界模型可作为环境模拟器
- ②**超长轨迹学习**：完成任务时长每7个月增加1倍，长任务能力关键在长轨迹学习，长轨数据、长轨奖励、长轨训练和记忆均存在挑战
- ③**经验Scaling（自进化）**：“扩大探索空间、创造无限经验、迭代内在理解” 闭环迭代是实现自主进化的核心
- ④**群体智能**：多智能体在并行效率和多路径探索等方面存在优势，联合优化多智能体协作，提升探索能力，在智能体网络中实现社会化
- ⑤**对算力底座的影响**：面向多环境交互、长轨迹、多模态等特点，系统需支持训-推-交互分离，异步、流水编排，算法需优化减少异步的精度损失

建议：聚焦agentic模型，交互环境，反馈系统，评测，agent算力底座



1

Agentic大模型

- 业界agent的发展趋势，大模型厂商的agent布局思路？
- Agentic AI对大模型当前能力和下一跳能力的诉求？
- 如何高效编排任务，部署多agent？长短记忆融合？

2

Reward scaling

- 如何reward scaling，提高RL上限？
- 如何构建支持多模态，多工具和不同颗粒度反馈信号的Reward系统
- 如何通过和环境交互产生reward信号？

3

评测

- 如何评价Agent的核心能力？复杂度，准确率，可靠性？
- 如何构建准确评估agent能力的测试集？
- 如何评估Agent产品在真实互联网场景的体验？

4

环境交互&工具使用

- 如何构建可靠，真实，多元的环境让agent交互？
- 模型在预训练和后训练阶段获得使用工具的通用能力？
- 环境交互和工具调用下的推理负载特征？

5

算力底座

推理算力需求剧增，需要高效推理；复杂流程，需要高效编排；高性价比，动态调度计算资源提高利用率

- 如何高效调度资源，提高资源利用率？去中心化？
- Agentic RL如何提高端到端系统吞吐？多轮交互，超长序列，长尾严重，如何异步RL和分布式RL

Agent scaling law

Agentic系统需要数量+reward+context的scaling

Agent数量scaling

Context scaling

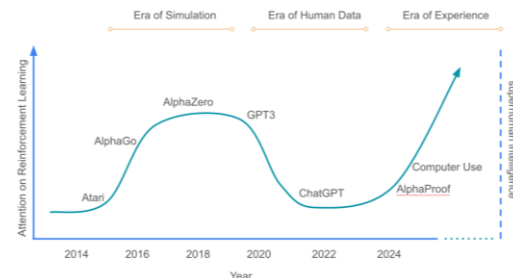
长轨迹+记忆scaling

模型能力scaling

反馈reward scaling (学习)

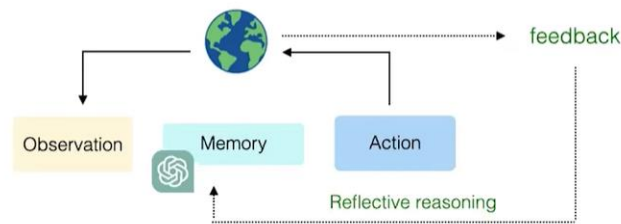
多环境/工具：交互持续获得高质量数据

- Agent数量越多，能使用的工具数量越多
- Agent数量越多，能体验的环境越多
- agent在环境中交互持续生成动态的数据给agent学习
- 迭代是由和现实环境验证的速度决定的，能快速，准确验证的环境，模型能力就提升的越快



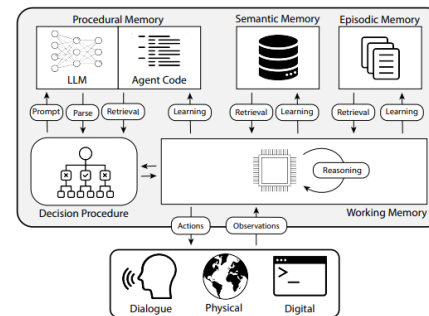
Reward：获得准确丰富的学习信号

- 核心就是数据不单纯以人类反馈为中心，到亲身经历学习+人反馈的二手信息的范式
- 行动和认知都需要在环境中grounding
- 三大场景：1. 搜索；2. computer use；3. coding==验证成本低，交互或者action的结果可以快速验证且试错成本低，同时方便compute scaling



长轨迹+记忆：在时间维度scaling，提升能力

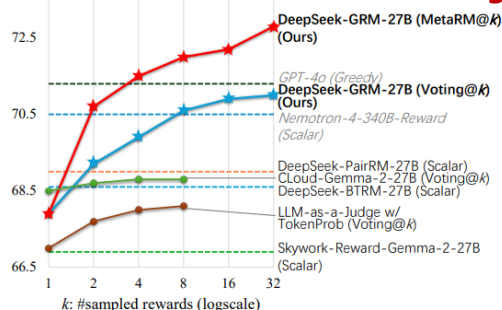
- Stateless 到stateful的转换，从单点的交互到持续的交互体验
- 记忆不同交互周期的数据，利用多层级存储系统高效读写和学习



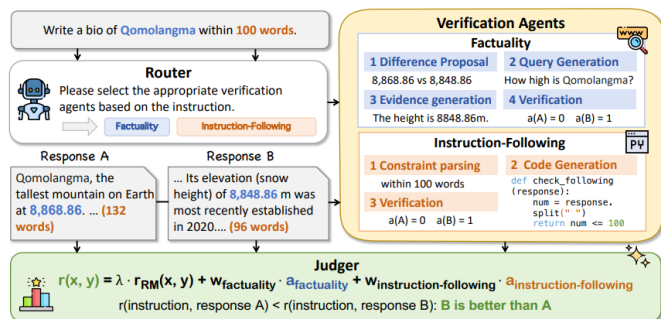
Agent scaling-Reward scaling: 提升reward质量, 为RL推理数据提供精确, 稠密的奖励信号, 从而能实现更强的RL scaling

业界趋势

1. 通用生成Reward: Pointwise的生成RM能更好scaling;更多reward计算提升精度



2. Reward agent: 用agent模块化集成多样工具, 提供不同类型的奖励信号



3. 世界偏好reward: 如Qwen的WorldRM用人类在环境交互中的反馈信号

4. 构建答案+格式+工具执行的多样outcome reward, 如微软agentic模型

$$\text{Tool Execution Reward} = \frac{\text{Tool}_{\text{success}}}{\text{Tool}_{\text{total}}}$$

Reward系统scaling

观点1: RM的算力不只是一次前向计算, 也要算力scaling

- 原来只强调要有“爱因斯坦”的RM, 但更需要其投入大量时间思考; 从现在的快思考打分, 借助模型的reasoning能力, 实现慢思考打分
- 包括更大的RM, 多个RM打分, 多次前向等

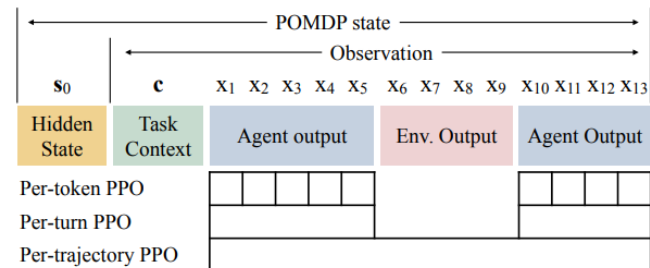
观点2: 丰富的环境交互提供更稠密的精准reward信号

- Agent模型有工具使用的能力, 可以借用工具对推理生成进行评价
- 如何environmental scaling 多样性多类别的奖励信号
- Reward系统随真实目标和环境动态变化

观点3: extrinsic+intrinsic奖励信号

- 混合奖励信号
- 短期和长期目标的奖励
- 自学习/自演进的奖励信号?

观点4: token/轮次/完整生成轨迹, 不同层级的奖励信号



子任务和全局目标的奖励; 可以对任务多阶段构建评价, 既是结果奖励也是过程奖励, 提供更稠密的奖励信号

Agent scaling-基于互联网的交互环境

超级APP是互联网原生，环境资源最丰富，交互自然，且自带反馈信号的环境，将是agent交互合适的digital环境

- 互联网是一个很合适的agent环境，可扩展，有语义，可互动，动态，且真实【社交媒体做交互环境】【原生就有reward】其实环境，reward，agent，最需要的是超级APP
- 通用级别的超级Agent，更能落地的产品形态是在解决通用检索的功能基础上，还能作为背后千万个垂类Agent的统一调用前端。简单而言，也就是一个AI Agent时代的超级入口和“应用商店”

Ship a: **3P Confidential** by 2026

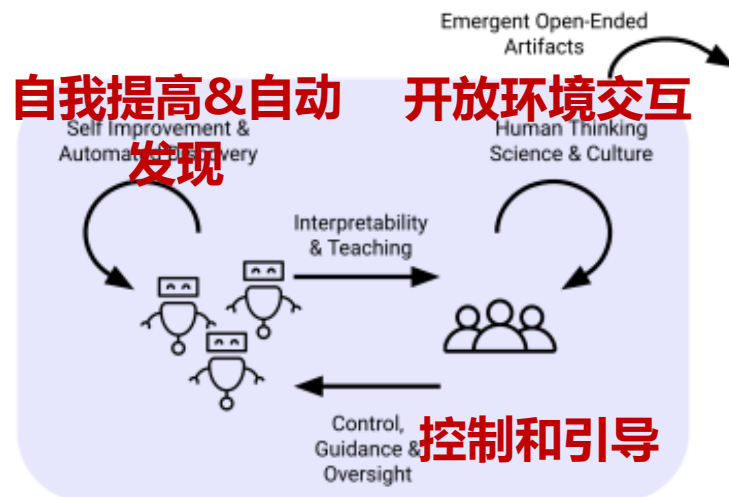
Today, ChatGPT is in our lives through existing form factors — our website, phone, and desktop apps. But our vision for ChatGPT is to help you with all of your life, no matter where you are. At home, it should help answer questions, play music, and suggest recipes. On the go, it should help you get to places, find the best restaurants, or catch up with friends. At work, it should help you take meeting notes, or prepare for the big presentation. And on solo walks, it should help you reflect and wind down. We want ChatGPT to be: **3P Confidential**

3P Confidential The best AI is the one that's right there with you.



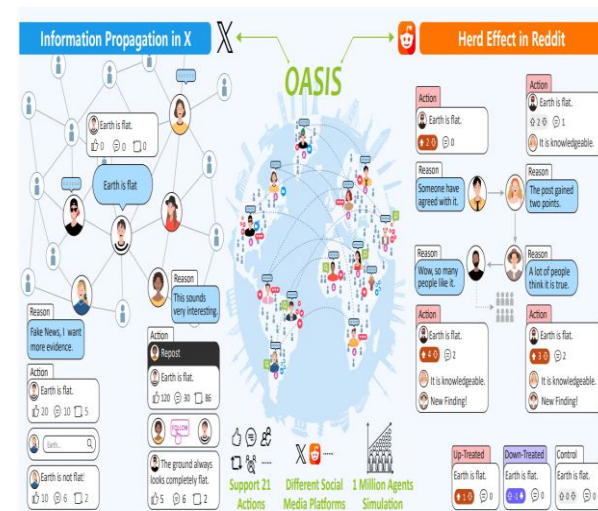
deepmind认为**开放环境**是超人类智能体ASI的必须

- 通过open-ended play实现通用能力的agent
- 新的环境中包括没有见过的，学习到隐空间中的关系；**能生成新的任务**，自我适应和优化，完成各种任务
- 在环境中，学习到如何在世界行动，并了解行为如何影响世界
- 生成新的知识/能力**: 假设，洞见，人类数据所没有的创造性输出



游戏环境: agent群体协作构建模型的交互环境，让模型在交互和反馈中学习和创造

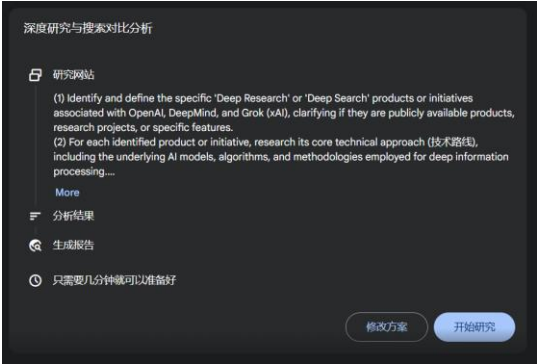
- 设置奖励函数；问题有泛化和上下文窗口
- 让agent在网络上发帖，留言，讨论；
- 随着多智能体规模逐渐扩大，之间的交互变得频繁且多样化，这些群体行为和对应的反馈，进而涌现出群体智能。



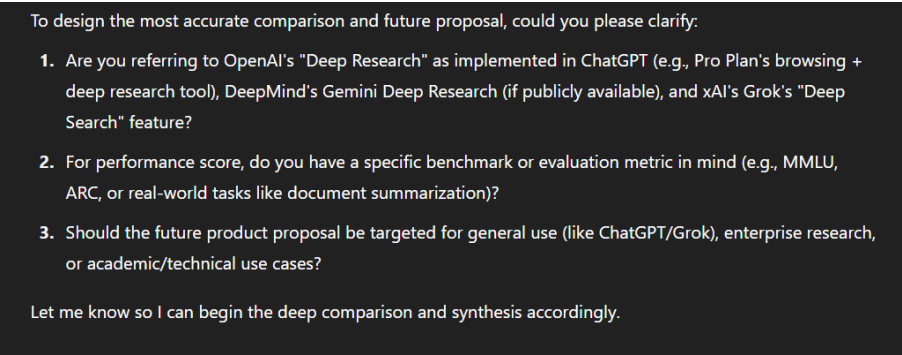
25H1: deep research本阶段代表性的Agent 产品形态

和传统 LLM Search 产品相比，Deep Research 是迈向 Agent 产品雏形的一次跃迁，可能也将成为具有阶段代表性的经典产品形态。

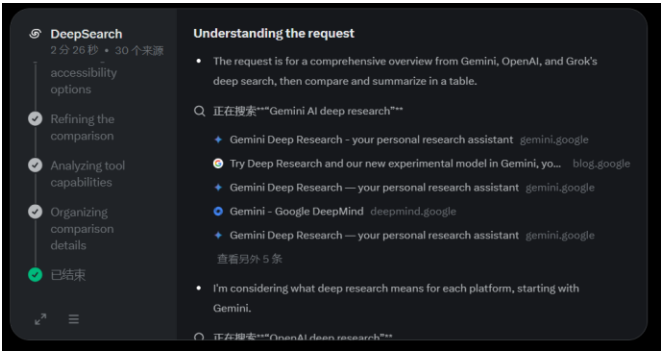
谷歌：让用户修改执行流程和内容



Openai：显式提问，明确任务和用户具体需求



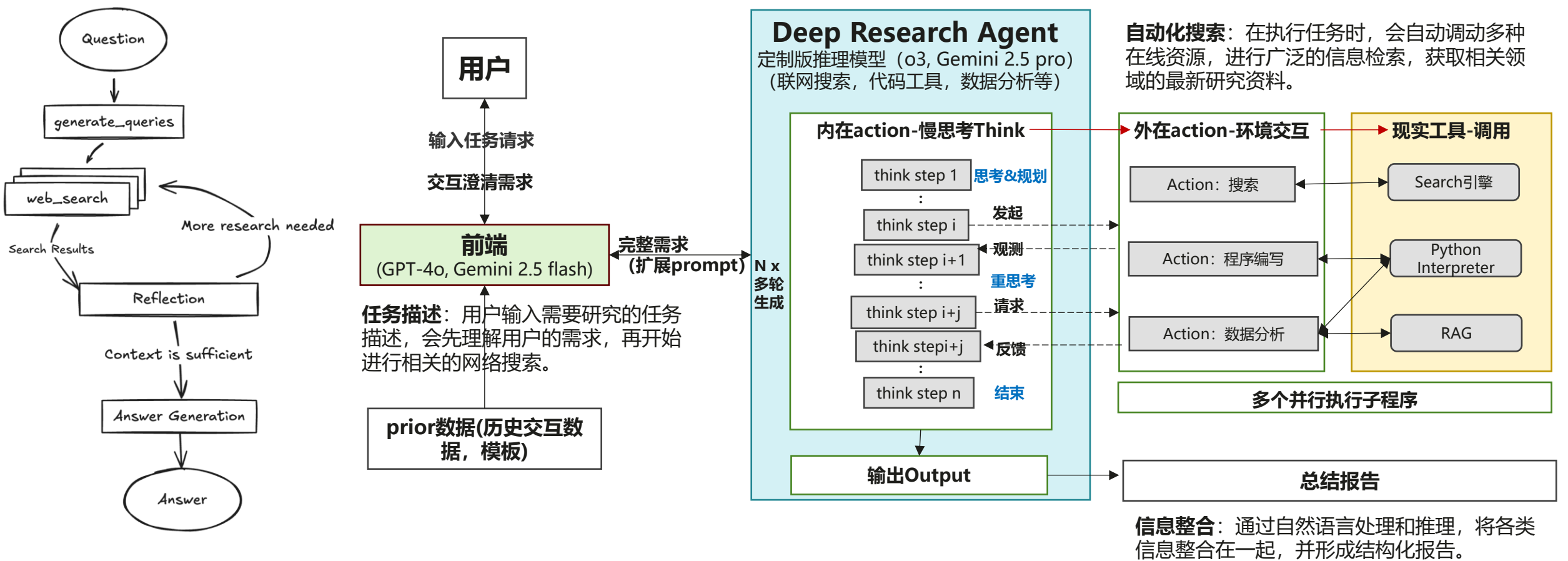
Grok：理解用户任务，直接生成计划



	谷歌Deep research	OAI Deep research	xAI Deep Search	下一代猜测
模型	Gemini 2.5 pro	浏览器、python工具增强的o3模型	Grok 3	原生agentic模型；多模型系统兼备快慢思考，多模态，工具使用和长context RAG
技术路线	模型+ReAct工作流调用工具+RAG	模型+browser/Python tools	模型+tool use, code interpreter和 web access	agentic模型和框架
上下文长度	1M	200k	128k	>1M
输入	文，文件	文，图，文件	文字	全模态+私有文件+自定义工具
输出	Text=报告; 语音总结	Text=报告	Text=报告	全模态+自定义形式+交互型输出
数据源	谷歌搜索; YouTube; google scholar	在线网页	推特X; 实时信息	互联网+私有数据+实时信息+
特点	+个人助手，交互：支持文件上传; 多语种; 语音总结 + 长context RAG agent	+ 质量最高：详细的完整报告; 结构化输出; 引用准确; 兼具深度和广度 + reasoning tool agent: 信息可靠	+ 及时信息：社交媒体信息; 简易报告 + 善于处理矛盾的信息	• 个性化：精准用户痛点 • 专业化+可靠性+可验证 • 跨硬件平台部署
生成速度/时延	数分钟	100 网页(25 来源, ~平均4 pages/source), 每个报告5-30min	快速生成，一般少于1min, 6-15秒生成报告	+ 低时延交互 + 弹性支持秒/分钟/小时/天级输出

Deep Research是基于ReACT范式组成的工具增强推理模型

- Deep Research 产品通过嵌入推理模型，使Agent 具备必要的推理能力。
- Deep Research 采用多次搜索和异步返回模式，在持续搜索过程中迭代和优化回复，从而输出更符合用户需求的内容，信息推理深度显著提升。通过自主计划、反思、行动，让 Agent 的能力可以完成复杂的任务。

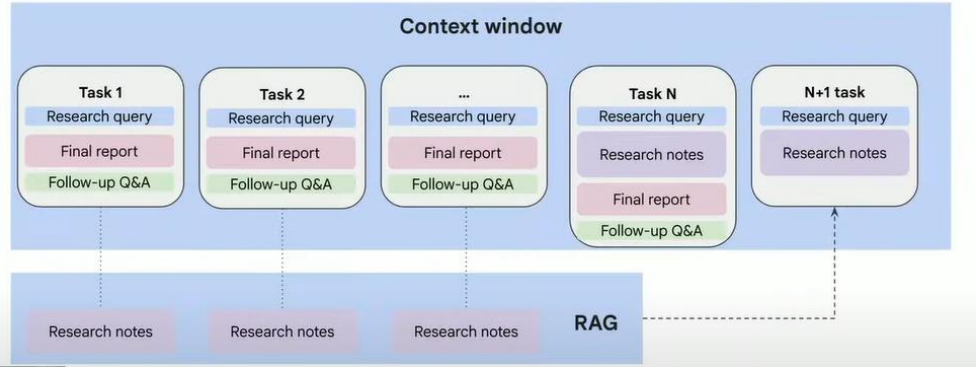


Deep Research 是本阶段代表性的Agent 产品形态

	谷歌Deep research	OAI Deep research	xAI Deep Search	下一代猜测
模型	Gemini 2.5 pro	浏览器、python工具增强的o3模型	Grok 3	原生agentic模型；多模型系统兼备快慢思考，多模态，工具使用和长context RAG
技术路线	模型+ReAct工作流调用工具+RAG	模型+browser/Python tools	模型+tool use, code interpreter和 web access	端到端训练
上下文长度	1M	200k	128k	>1M
输入	文，文件	文，图，文件	文字	多模态+自定义工具
输出	Text=报告; 语音总结	Text=报告	Text=报告	多模态+交互型输出
数据源	谷歌搜索; YouTube; google scholar	在线网页	推特X; 实时信息	互联网+私有数据+办公软件+APP
特点	+个人助手，交互：支持文件上传; 多语种; 语音总结 + 长context RAG agent	+ 质量最高：详细的完整报告; 结构化输出; 引用准确; 兼具深度和广度 + reasoning tool agent: 信息可靠	+ 及时信息：社交媒体信息; 简易报告 + 善于处理矛盾的信息	• 个性化：精准用户痛点 • 专业化+可靠性+可验证 • 跨硬件平台部署
生成速度/时延	数分钟	100 网页(25 来源, ~平均4 pages/source), 每个报告5–30min	快速生成，一般少于1min, 6-15秒生成报告	+ 低时延交互 + 弹性支持秒/分钟/小时/天级输出

- 上下文管理
- 多模态
- 互联网交互环境
- Agent评测
- Agent部署

例子：谷歌deep research的上下文管理



June 4, 2025

Connectors in beta for deep research (Plus, Pro, Team, Enterprise, Edu)

ChatGPT Team, Enterprise, and Edu customers globally can use connectors in deep research, as well as Pro and Plus users (excluding users in Switzerland, EEA, and the UK) to generate long-form, cited responses that include your company's internal tools.

- Supported connectors: Google Drive, SharePoint, Dropbox, Box, Outlook, Gmail, Google Calendar, Linear, GitHub, HubSpot, and Teams
- Combines internal + web sources for synthesis

Learn more about [Connectors in ChatGPT](#).

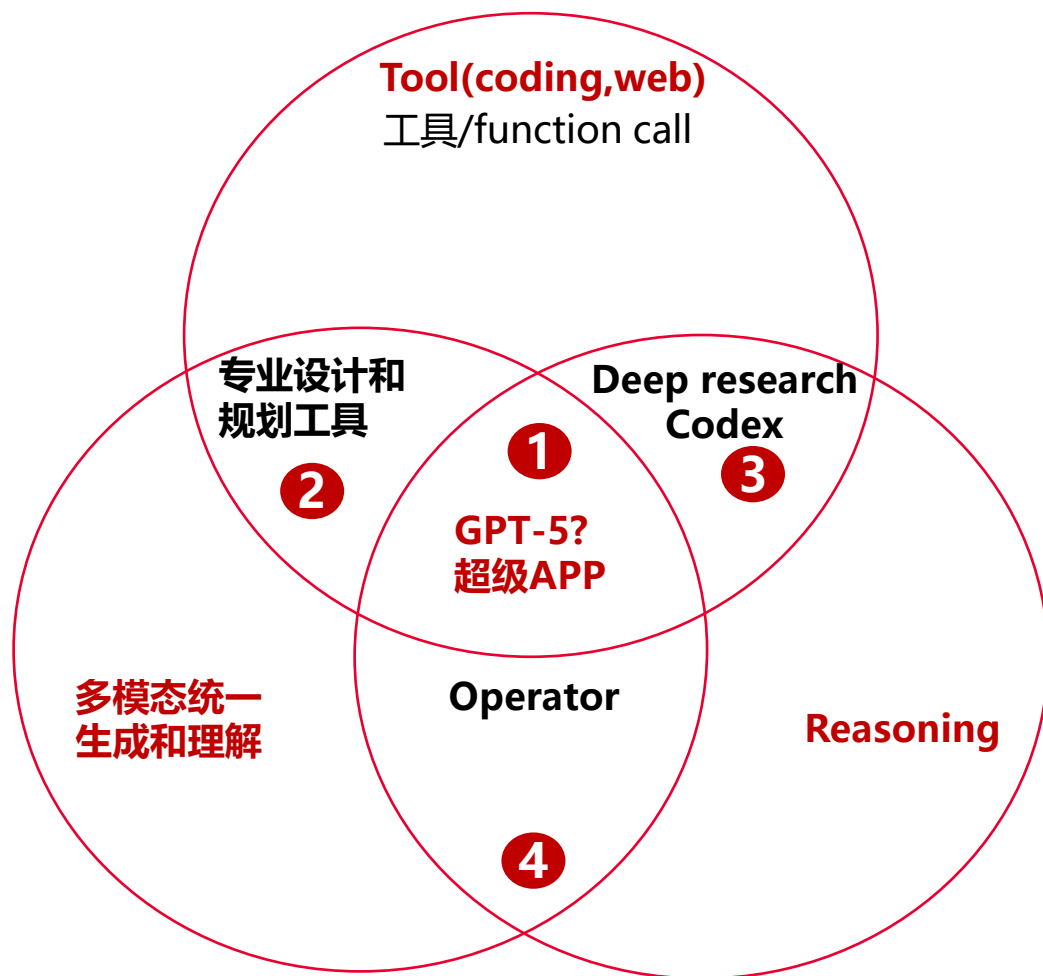
Custom connectors via Model Context Protocol (Pro, Team, Enterprise, Edu)

Admins and users can now build and deploy custom connectors to proprietary systems using Model Context Protocol (MCP).

- 办公软件套件
- 内部数据库
- 通过MCP自定义

25H2: 多模态+慢思考+工具调用(coding)复合能力

agentic基模能力复合提升，支撑不同应用场景



① 融合统一接口/超级APP-GPT5 将结合所有agents/产品

- 使用更多现实环境中的工具
- 快慢思考
- 多模态理解和生成

The next iteration of the Gemini app

- **Universal assistant**
Connects all of your Google stuff in a single place (with your permission)
- **Native everywhere you want it to be**
Mobile, desktop, glasses, car, TV, etc...
- **Personalized with your data**
If you give the permission and it is useful information
- **Proactive**
Stop making users do all the work

②

Tool integrated 多模态统一生成和理解

利用工具扩展处理多模态数据的能力；提高鲁棒性和可解释性；增强在专业领域的能力

③

Tool integrated reasoning

跟cot的区别就是在推理的过程中会使用工具，这样推理过程动态的添加了search, code, 各种定制化的API的输入，推理能力得到了进一步的增强

④

多模态快慢思考

- 在多模态理解的基础上，reasoning让模型能更好的planning, reflection, 从而可以更高效的完成任务和纠正错误

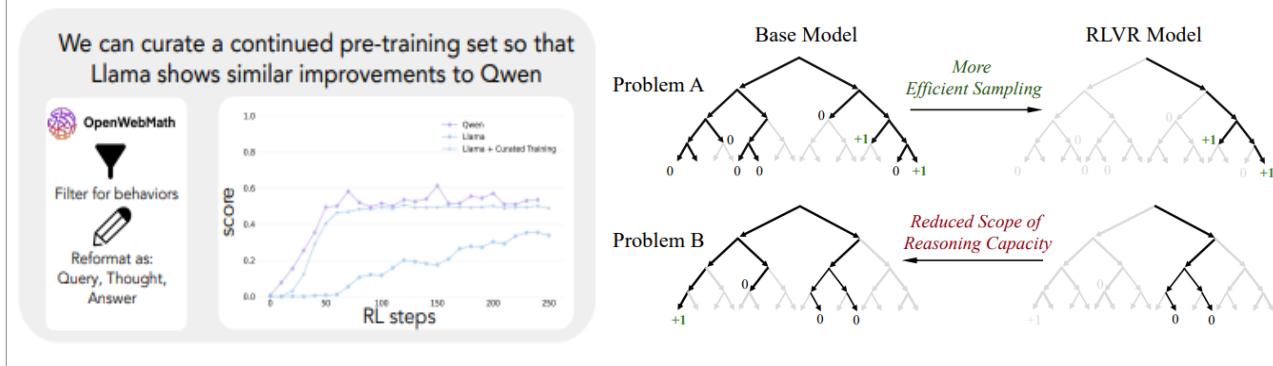
Agent能力的构建前移到预训练阶段，更好的预训练提高模型agentic性能

更强的基模GPT-4.5支持更好的tool use agent

Stronger reasoning on the horizon

GPT-4.5 doesn't think before it responds, which makes its strengths particularly different from reasoning models like OpenAI o1. Compared to OpenAI o1 and OpenAI o3-mini, GPT-4.5 is a more general-purpose, innately smarter model. We believe reasoning will be a core capability of future models, and that the two approaches to scaling—pre-training and reasoning—will complement each other. As models like GPT-4.5 become smarter and more knowledgeable through pre-training, they will serve as an even stronger foundation for reasoning and tool-using agents.

预训练压缩action space；继续训练可以提升RL训练上限；RL让模型偏好生成高reward的路径



deep research: o3+RL on browsing; codex: o3+RL on software engineering-> codex-1

Quote1: **Coding和视觉能力预训练，快慢思考后训练** 【O3-CUA-system card】 o3 Operator inherits o3's coding capabilities, combines o3/GPT-4o's vision capabilities with advanced reasoning through reinforcement learning

Quote2: **预训练阶段对reasoning能力构建也很重要**； OpenAI o3-mini was pre-trained on diverse datasets, including a mix of publicly available data and custom datasets developed in-house, which collectively contribute to the model's robust reasoning and conversational capabilities

Quote3: **端到端agent模型训练需要数据包含感知，推理，记忆，行动的信息**； 【字节GUI】 training an end-to-end agent model demands data that integrates all components in a unified workflow, capturing the seamless interplay between perception, reasoning, memory, and action. Such data, which comprise rich workflow knowledge from human experts, have been scarcely recorded historically

Quote 4:**通过仿真环境构建预训练数据集** Leveraging hundreds of virtual machines, UI-TARS explores diverse real-world tasks based on constructed instructions and generates numerous traces. Rigorous multi-stage filtering—incorporating rule-based heuristics, VLM scoring, and human review—

Quote 5: **训练数据排序，从局部元素到全局界面**We adopt a bottom-up data construction approach, starting from individual elements and progressing to holistic interface understanding. By focusing on small, localized parts of the GUI before integrating them into the broader context, this approach minimizes errors while balancing precision in recognizing components with the ability to interpret complex layouts

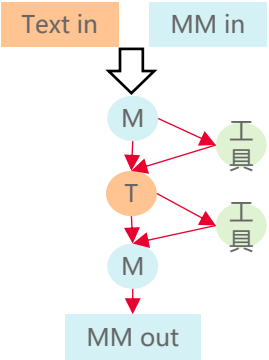
Quote 6: **通过Continual Pre-training让模型学会GUI基础能力的知识**： we utilize the full set of data described in § 4, excluding the reflection tuning data, for continual pre-training with a constant learning rate. This foundational phase allows the model to learn all the necessary knowledge for automated GUI interaction, including perception, grounding, and action traces, ensuring robust coverage across diverse GUI elements and interactions.

25H2Agentic模型的关键是融合多模态，慢思考和工具调用三个基础能力，产生复合效果

多模态+慢思考+tool use

多轮带工具调用的思维链

GPT5, Gemini 3

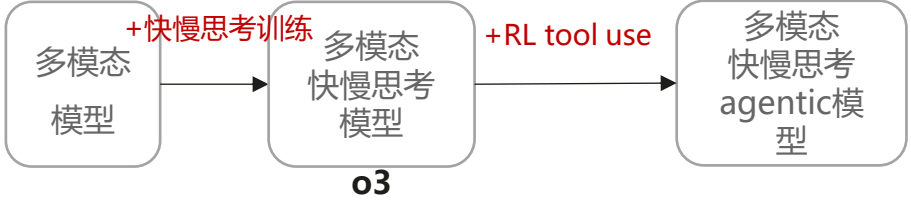


+工具调用：生成阶段有<tool>...</tool>，工具能够辅助多模态理解和生成；多模态理解提升工具使用能力

路径1

单模型具备
多模态
agentic能力

路径1



路径1 (推荐) 多模态+慢思考+工具调用方案：推荐在多模态快慢思考模型上添加工具调用方式训练

优点：模型性能上限高

- 1、先训练多模态快慢思考模型再引入工具能力，可扩展
- 2、最大化多模态模型的能力

路径2.1



路径2

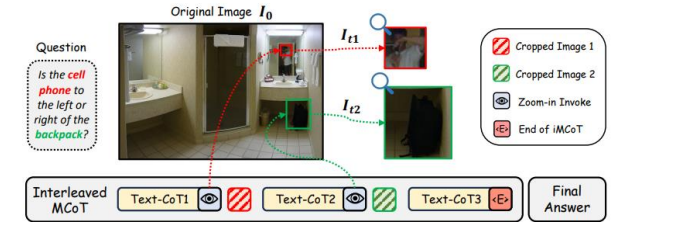
优点：训练要求低；针对场景灵活组合模型，工具和工作流；可独立迭代
缺点：

- 1、系统集成复杂，不通用，现有agent效果不好
- 2、时延偏大，影响用户体验

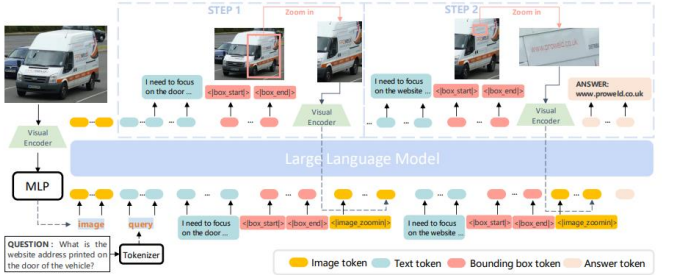
多模态+tool: 学会利用工具处理图像增强多模态能力, 多模态生成将作为模型思考的内生工具

1. 学会利用工具在CoT中包含图像

小红书DeepEyes: 调用工具处理图像, 后输入回到模型, 与文本共同进行下一步推理

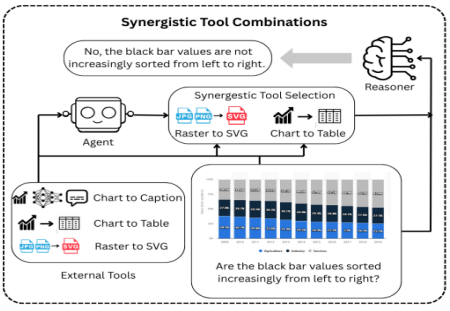


Chain of Focus: CoT中有对图像缩放和裁剪

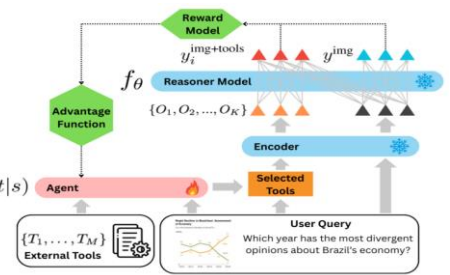


2. 学会根据问题选择合适的工具

Agent先学会调用和合成工具组合

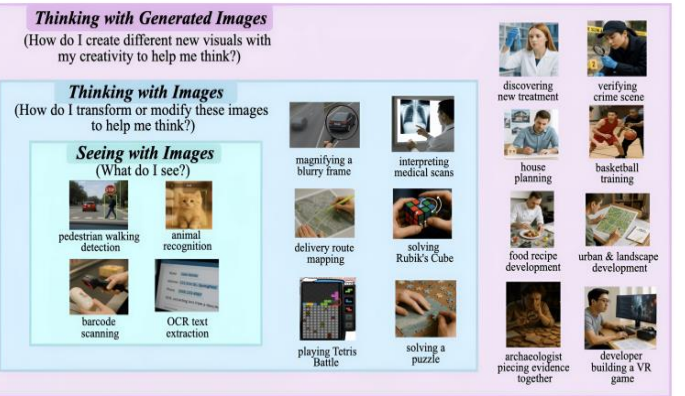


VisualToolAgent: 通过RL训练让agent学会根据问题选择合适的工具辅助VLM进行推理

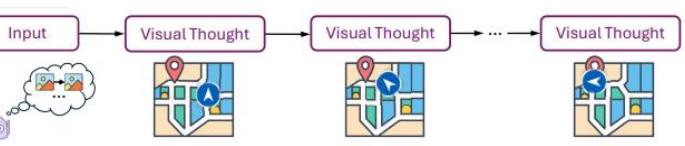


3. 结合模型生成能力修改图像, 增强思考

GAIR: 用生成图像思考

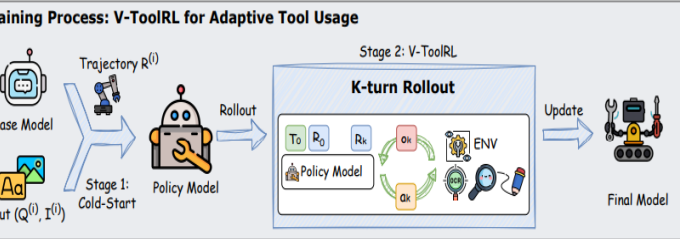


Visual Planning: 用纯视觉表示进行规划



技术1: 视觉工具RL训练框架

OpenThinkIMG框架: 可选的SFT+多样视觉工具环境+多轮rollout



技术2: 合适的reward信号

只有准确输出, 使用工具才有奖励
$$R(\tau) = R_{acc}(\tau) + R_{format}(\tau) + \mathbb{I}_{R_{acc}(\tau) > 0} \cdot R_{tool}(\tau),$$

对合适工具选择的奖励

$$r_j = \begin{cases} +1 & \text{if } y_j^{img} \neq y^* \text{ and } y_j^{img+tools} = y^* \text{ (tools help)} \\ -0.5 & \text{if } y_j^{img} = y^* \text{ and } y_j^{img+tools} \neq y^* \text{ (tools hurt)} \\ 0 & \text{if } y_j^{img} \neq y^* \text{ and } y_j^{img+tools} \neq y^* \text{ (no change)} \\ +1 & \text{if } y_j^{img} = y^* \text{ and } y_j^{img+tools} = y^* \text{ (tools neutral)} \end{cases}$$

技术3: 多层颗粒度的图像处理数据

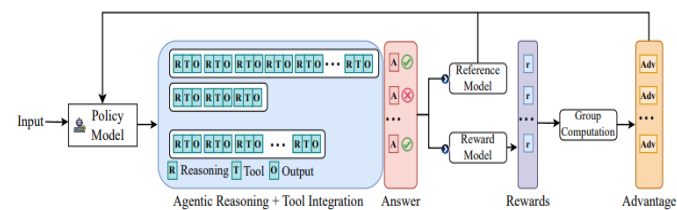
SeedEdit3.0: 让模型听懂指令, 理解不同任务差异, 在图像中区分需要操作和保持的部分;



reasoning+tool: 利用工具与环境交互，增强模型reasoning能力

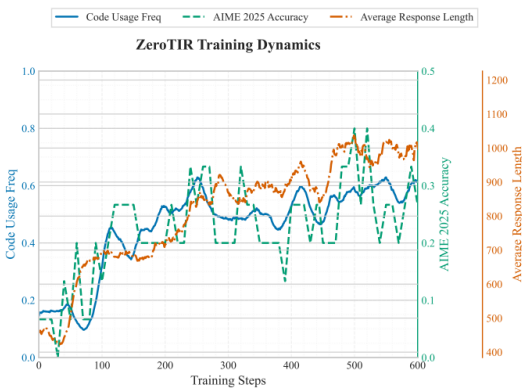
将文本生成与工具和环境查询交织在COT

微软ARIST: 使用结构化的提示模板，将输出分为四个部分: (1) 内部推理 (<think>...</think>), (2) 工具或环境查询 (<tool_name>...</tool_name>), (3) 工具输出 (<output>...</output>), 以及 (4) 最终答案 (<answer>...</answer>)。



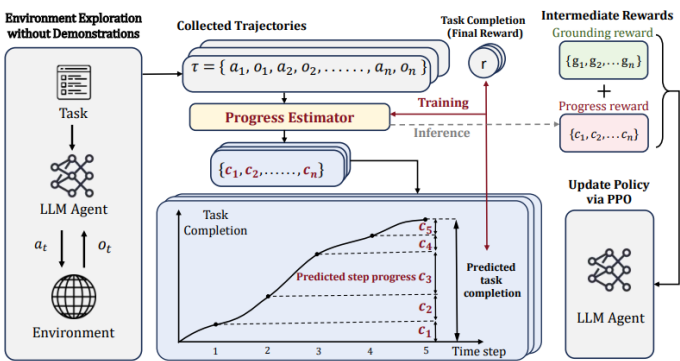
随着RL训练步数提高coding执行频率

Agent RL scaling law: 训练步数的增加会导致自发代码执行频率、平均响应长度和最终任务准确率



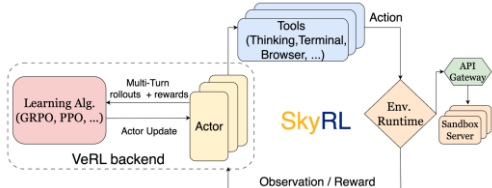
工具和环境交互获得更稠密的奖励信号，提高模型长任务学习能力

SPA-RL: 用工具验证中间步骤的正确性，提供分步骤reward，提升长任务的完成率和可重复性



技术: 包含tool use的异步RL训练框架

SkyRL: 引入agents层扩展了VeRL: (1) 高效的异步多轮rollouts, (2) 通用工具使用, 以及 (3) 通用和可扩展的环境执行



技术: 大模型作为RL训练环境

ZEROSEARCH: 利用大模型自身知识，将其作为搜索引擎，让LLM学习搜索策略，无需外接搜索环境

You are the Google search engine.
Given a query, you need to generate five [useful / noisy] documents for the query.
The user is trying to answer the question: [question] whose answer is [ground truth].
Each document should contain about 30 words, and these documents should contain [useful / noisy] information.
Query: [query]
[Useful / Noisy] Output:

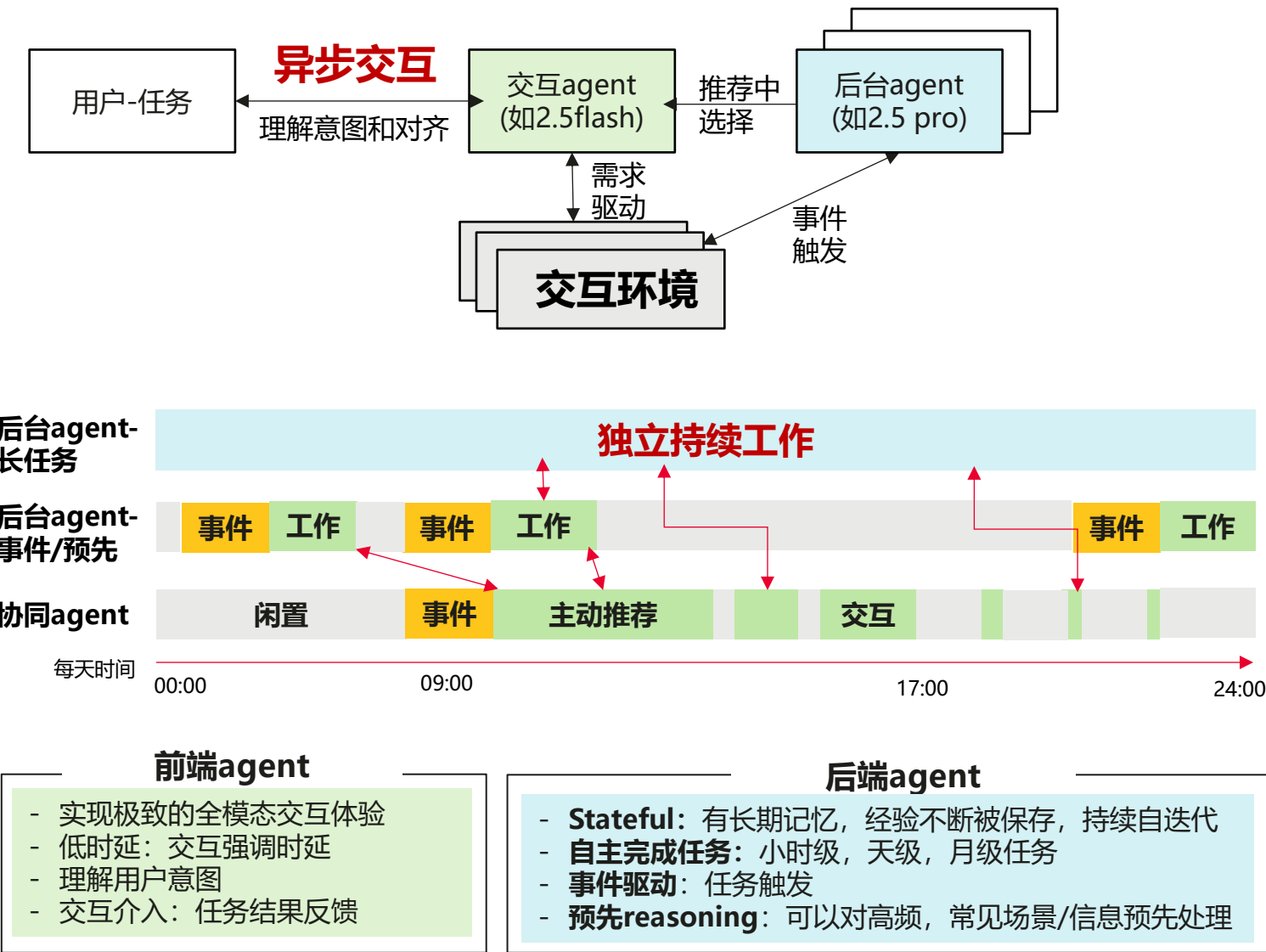
技术: 奖励配置策略

ToolRL: 探索了四个关键维度的各种奖励配置: (1) 奖励类型 (奖励哪些方面), (2) 奖励尺度 (奖励多少), (3) 奖励粒度 (奖励信号的详细程度), 以及 (4) 奖励动态 (奖励如何随时间演变)

2026: >24小时的长任务

趋势：交互从被动“问-答”，转变为“主动推荐-用户选择” -> Agent从 Copilot转为Autopilot

Agent要能够异步自主地执行更长任务



Agent能够独立完成更长的任务

子趋势1：人类时间有限不可扩展
-> **agent需持续扩展工作时长**

子趋势2：人类交互频率有限
-> **agent需更自主地工作**

子趋势3：人类能力有限
-> **agent需完成更复杂任务**

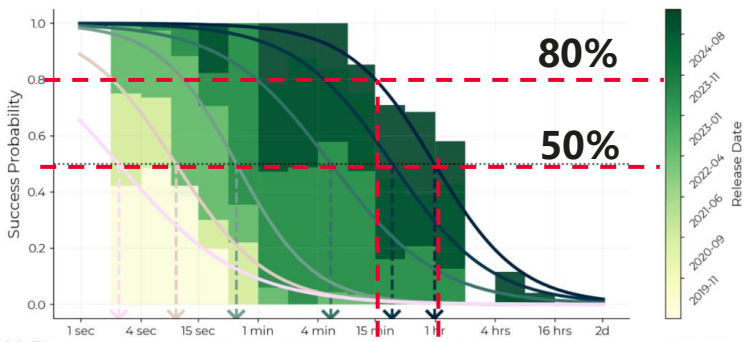
超长时任务：Agent独立完成任务的时长每7个月增加1倍，将处理更复杂任务，创造更大价值

- Agent长任务能力依靠：将复杂问题分解为子任务，决定用什么工具解决任务，可靠地调用合适工具和输出对齐
- 提升超长任务能力需要**长轨迹学习**，关键挑战在于：**长轨迹数据**，**奖励稀疏性**，**效率以及长任务记忆**

当前不足：成功率随任务的长度衰减

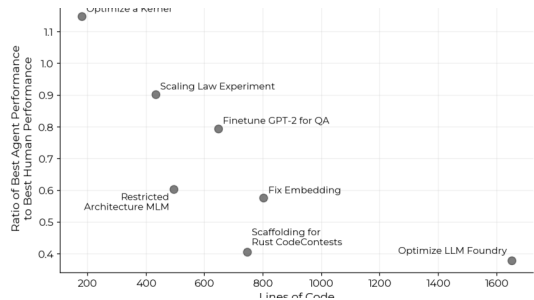
80% 成功率下只能完成 15 分钟的任务

模型“半衰期”



横轴：任务长度-人类专家完成任务的时间

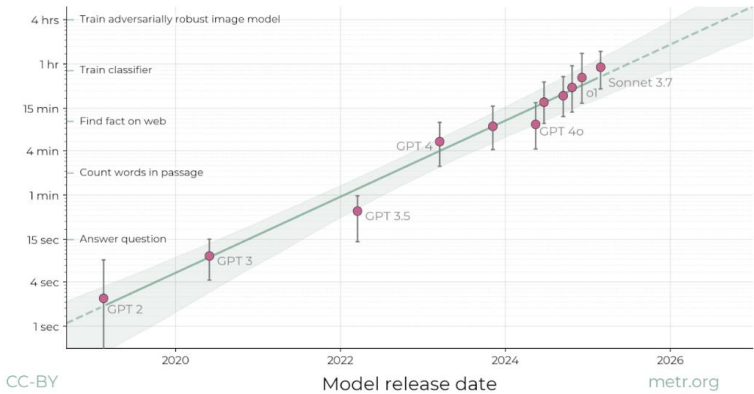
越复杂的任务，agent和人类专家的差距越大



横坐标：环境复杂度，代码行数

趋势：完成任务时长每7个月增加1倍

完成任务的时长不断提升



50%准确率时长预测：目前普遍：0.5-1小时

2026年底可执行任务时长：>1天

独立任务时长增加，改变agent交互方式

- 从Copilot到Autopilot，减少实时监督/交互
- 从问答，到“推荐结果-用户选择”的范式
- 通过agent扩展人类工作时长

“ Claude Opus 4 offers truly advanced reasoning for coding. When our team deployed it on a complex open source project, it coded autonomously for nearly **seven** hours—a huge leap in AI capabilities that left the team amazed.

自主长任务能力解锁新应用场景

分钟级-搜索

15秒=QA问答

10分钟=在互联网找准信息

3-30分钟 DeepResearch形成万字研究报告

小时级-Coding

1小时=训练分类器

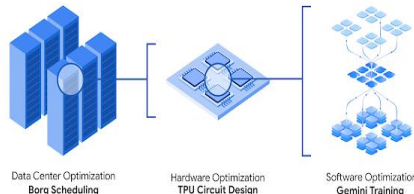
4小时=训练图像模型

Model	Made Submission (%)	Valid Submission (%)	Above Median (%)	Bronze (%)	Silver (%)	Gold (%)	Any Medal (%)
AIDE							
o1-preview	98.4 ± 0.4	82.8 ± 1.1	29.4 ± 1.3	3.4 ± 0.5	4.1 ± 0.6	9.4 ± 0.8	16.9 ± 1.1
gpt-4o-2024-08-06	70.7 ± 0.9	54.9 ± 1.0	14.4 ± 0.7	1.6 ± 0.2	2.2 ± 0.3	5.0 ± 0.4	8.7 ± 0.5
llama-3.1-405b-instruct	46.3 ± 2.9	27.3 ± 2.6	6.7 ± 1.4	0.0 ± 0.0	1.3 ± 0.7	1.7 ± 0.7	3.0 ± 1.0
claude-3.5-sonnet-20240620	68.9 ± 3.1	51.1 ± 3.3	12.9 ± 2.2	0.9 ± 0.6	2.2 ± 1.0	4.4 ± 1.4	7.6 ± 1.8

MLE-Bench每24小时推理的成本是\$2812，奖金是\$25K；O系列：>16.9%，实现奖金>成本

天级-AI R&D

独立开展科学研究，
数据中心优化，
软硬件的优化



...

超长轨迹数据： 依赖模型和行业专家标注获得高质量SFT长轨迹数据和RL数据

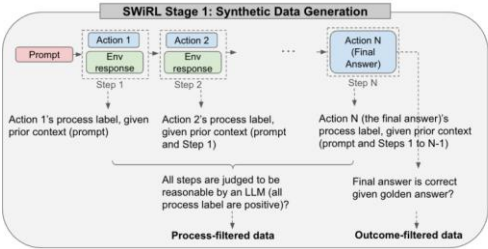
- 拥有用户行为数据的企业无需投入巨额预算合成数据，可通过Agentic learning打造独特优势

高质量长轨迹SFT数据合成：长度，规模，质量，多样性

长度：反复让模型多轮调用工具

挑战： 现有长度有限且不涉及多轮工具调用

- 给模型配上工具，每一步模型可以选择调用工具或者生成最后答案，反复prompt模型生成多步的轨迹
- 过滤只需要几步就能回答的简单的问题，控制最小步数，从而生成长轨迹



【斯坦福 SWiRL 25.04】

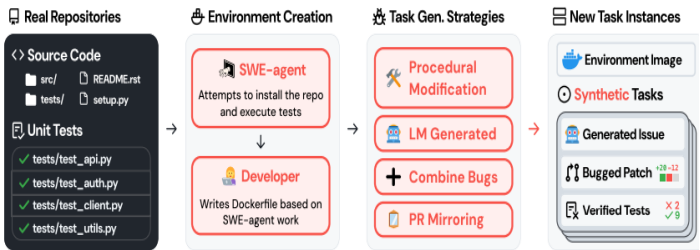
质量：利用业务专家知识构建模板和数据流程

Retool: OpenThoughts获得数学推理数据->人类专家和R1模型过滤数据->高质量文本推理数据->用专家设计的prompt模板将手动计算步骤转为代码执行->格式验证->专家验证答案->获得代码增强的长轨迹数据

规模：基于环境，模型生成扩展任务

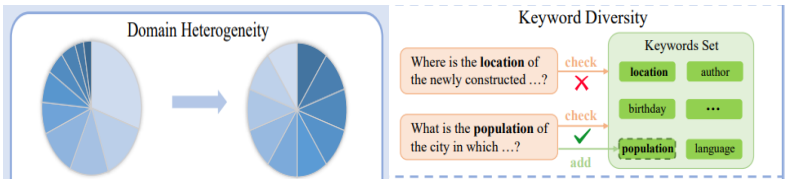
思路： 扩展环境易于扩展数据，从创建数据+调试环境->到先设置环境，创建任务路径数据的方式

方法： 从真实的代码仓库出发，为给定仓库设置执行环境，保证了测试可运行；然后系统化的插入bug，生成对应的修复操作和验证路径，形成50k个任务实例



【斯坦福SWE-smith 25.05】

多样性：SimppleDeepSearcher: 在真实的web环境使用多样性感知query采样



RL数据：复杂任务识别

专家基于agent长任务失效模式构建复杂任务

- 针对agent常见失效类型：推理错误，搜索结果错误，幻觉和标签模糊，导致agent提前中止任务

模型组合多样的子任务构建复杂长任务

伯克利AgentSynth：在电脑操作环境构建多类别的子任务集，组合子任务构建成复杂任务，多级组合不断增加难度

高质量RL数据： RM得分分布应具备多样性，有别于传统LLM RL只需模型适配，agent需要结合模型+环境适配才能筛选出高质量RL数据

依赖稳定高质量检索结果，并保证与RL数据的适配性

环境交互面临不确定性，需定期更新数据并强化微调

超长轨迹奖励：奖励稀疏被放大，未来需利用外部工具和内生自验证提供更稠密的奖励信号

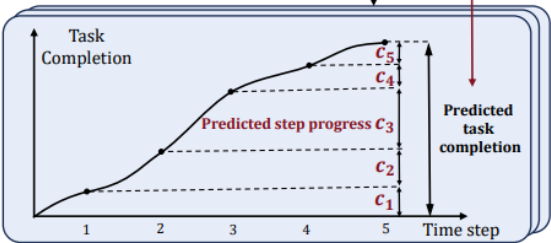
- OpenAI: “完美地在特定领域实现产品化的评分和任务生成，这可能 2 年内最需要解决的问题。”
- 完整长轨迹的ORM，到子任务的ORM或者ORM+PRM，提供更丰富的轨迹级奖励，更细的过程稠密奖励

利用工具与环境交互，提供更稠密的外部奖励

-> 增加长任务链中的反馈密度，减少中间步骤的错误，提高完成率和可重复性

SPA-RL: stepwise的reward

问题：等超长任务执行完再获得反馈信号效率太低
方法：用工具验证中间步骤的正确性，根据任务完成度分出多个关键节点，对关键节点的完成度提供reward；检测到错误并纠正给额外的奖励



苹果LOOP: APPWorld环境里token,轮次, 轨迹的多层级奖励

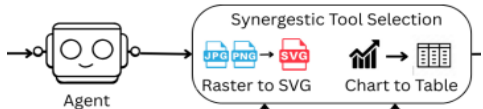
s0	c	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13
Hidden State	Task Context	Agent output				Env. Output				Agent Output				
Per-token PPO														
Per-turn PPO														
Per-trajectory PPO														

-> 增加长任务链中的反馈类型，对工具的选择，组合和使用提供奖励信号，提高使用工具的效率

VisualToolAgent: 调用和组合工具的reward

问题：agent学会多个工具混合多轮调用

方法：



$$R(\tau) = R_{acc}(\tau) + R_{format}(\tau) + \mathbb{I}_{R_{acc}(\tau) > 0} \cdot R_{tool}(\tau),$$

奖励
输出
准确

奖励
输出
格式

只有准确输出，使用工具才有奖励

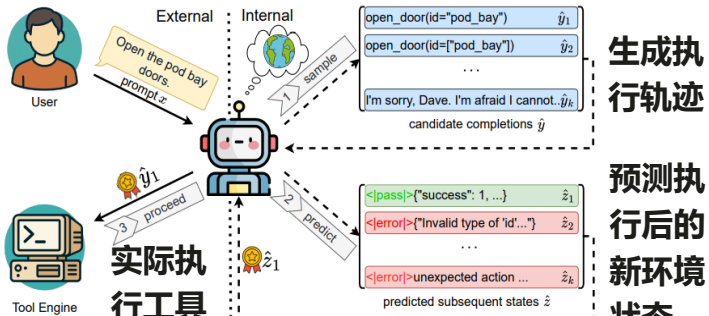
奖励
使用
工具

对选择工具后的不同效果设置合适的奖励尺度

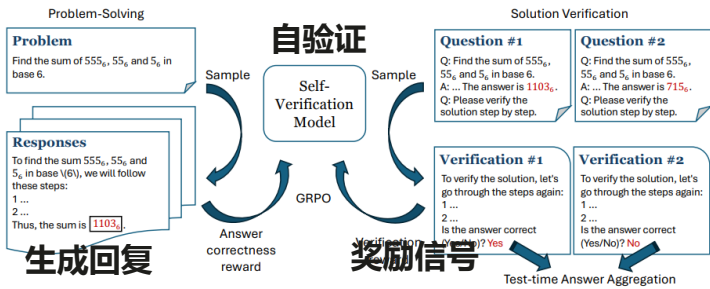
$$r_j = \begin{cases} +1 & \text{if } y^{img} \neq y^* \text{ and } y_j^{img+tools} = y^* \text{ (tools help)} \\ -0.5 & \text{if } y^{img} = y^* \text{ and } y_j^{img+tools} \neq y^* \text{ (tools hurt)} \\ 0 & \text{if } y^{img} \neq y^* \text{ and } y_j^{img+tools} \neq y^* \text{ (no change)} \\ +1 & \text{if } y^{img} = y^* \text{ and } y_j^{img+tools} = y^* \text{ (tools neutral)} \end{cases}$$

模型自验证，提供更稠密的内部奖励

-> 提供稠密、轨迹级信号，将环境模型与策略模型合一，减少对外部的依赖，并提供更丰富，更连续的信号



【25.06 Cohere: Self-Verification Sampling for Tool Use of LLMs】



【25.06 Skywork AI: Incentivizing LLMs to Self-Verify Their Answers】

超长轨迹训练效率：算法分解任务并提高rollout样本效率

- 天级，周级的rollout几乎无法实操，需要拆解长任务，在相对短的子轨迹高效rollout
- 核心是利用高效算法：分解任务自适应调度，减少rollout，聚焦有用样本和样本利用效率

现有方法难以直接扩展到超长任务

吞吐瓶颈： 朴素的LLM推理-工具调用的串行方式效率低，无法高效并行，系统吞吐低导致采样量不足，无法探索

迭代瓶颈： 天级，周级的rollout时间过长，RL迭代轮次将严重受限，policy更新的间隔时间长

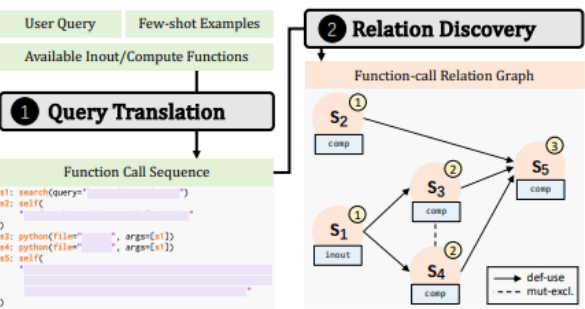
累积错误： 超长轨迹下，轨迹更容易受错误累积影响，rollout样本的质量随长度下降

稳定环境： 超长轨迹涉及多轮调用工具和环境交互，工具和环境的状态要保持一致，要高度稳定，否则单一轨迹内分布不一致

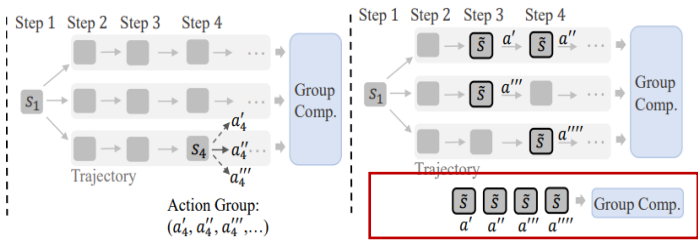
长任务拆解，子任务并行rollout

- 超长任务的子程序间有依赖关系
 - 子任务的重要性不同，有优先级
- 高效编排子任务，减少工具IO开销

蚂蚁LLMOrch



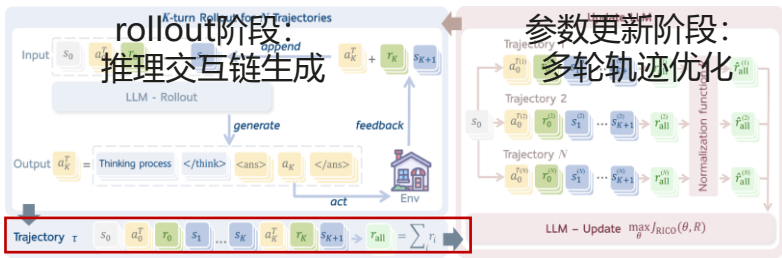
计算额外的优势，提高样本效率GiGPO：
两层优势融合，组合所有轨迹中相同环境状态的动作实现微观相对优势估计，提高样本效率



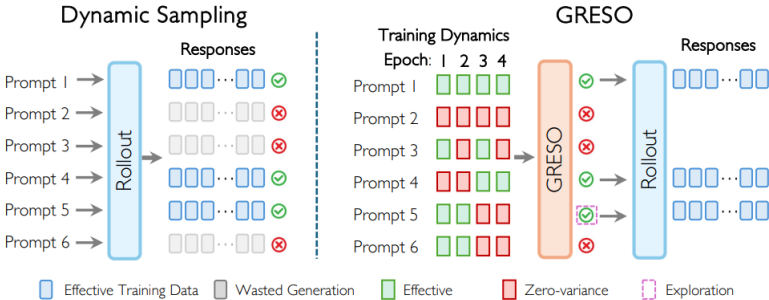
高效rollout算法提高效率

-> 思路1：不采用所有轨迹作优化，用抽样部分轨迹优化，可实现长远推理，同时保持计算效率

RAGEN: 依据重要性抽样-Max reward
Not All rollout are useful: 随机下采样



-> 思路2：在rollout阶段直接跳过，从源头减少计算：
GRESO实时跟踪每个prompt的历史奖励行为，对无效的prompt估算跳过概率，无需生成回答



2026: 用agent加速AI迭代+自演进
谁有最强agent, 谁就迭代更快, 迭代速度是核心护城河

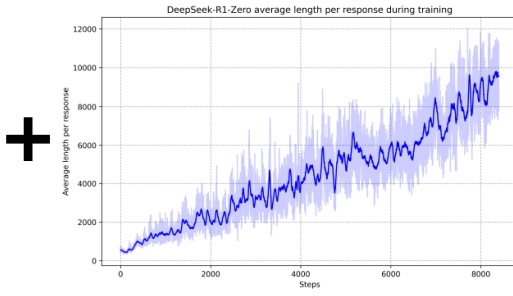
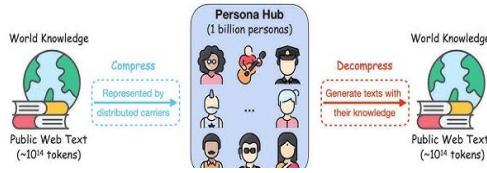
业界利用大模型和AI，不断加速整个技术栈和计算底座

新的scaling law，合成数据+ RL，模型迭代周期缩短

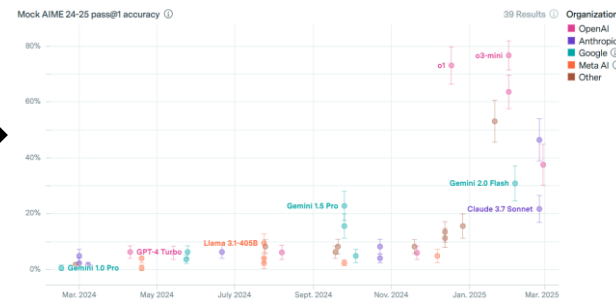
sakana.ai

Can LLMs invent better ways to train LLMs?

Scaling Synthetic Data Creation with 1,000,000,000 Personas



模型能力摸高和新涌现能力



自动优化软件栈，持续加速硬件，支持负载特征变化

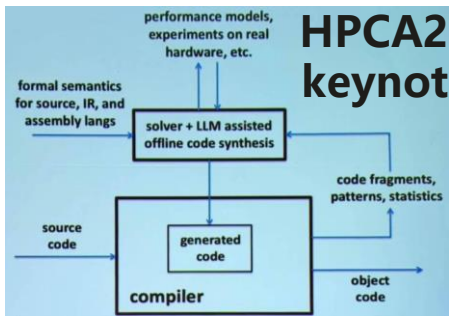
谷歌超过25%的新代码都是AI生成



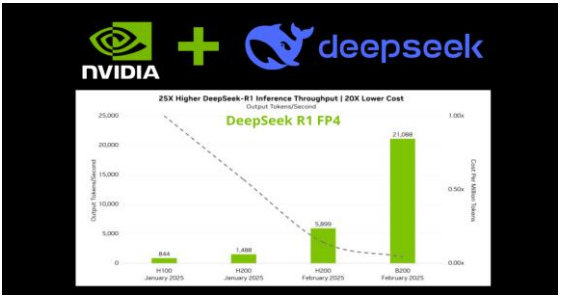
Automating GPU Kernel Generation with DeepSeek-R1 and Inference Time Scaling

Feb 12, 2025
By Terry Chen, Bing Xu and Kirithi Devlekar

+68 Like | Discuss (2)



更高效的kernel和推理软件栈->更高ROI



持续缩短新硬件的迭代周期

芯片仿真加速+工厂生产流水线加速和优化+光刻过程+计算光刻+芯片设计辅助

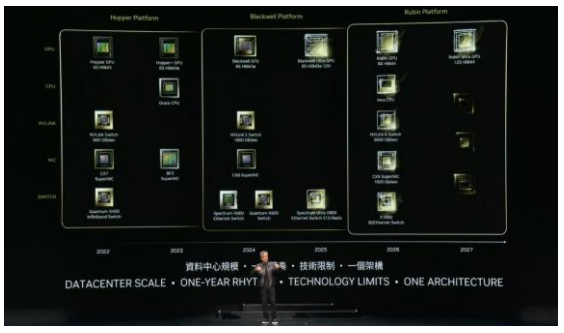


NVIDIA cuLitho
Accelerate computational lithography
NVIDIA cuLitho is a library with optimized tools and algorithms for GPU-accelerating computational lithography and the manufacturing process of semiconductors by orders of magnitude over current CPU-based methods.
Manufacturing computer chips requires a critical step called computational lithography – a complex computation – involving electromagnetic physics, photochemistry, computational geometry, iterative optimization, and distributed computing.
This computational lithography step is already one of the largest compute workloads in semiconductor production, necessitating massive data centers, and the silicon miniaturization evolution process exponentially amplifies the computation requirements over time.

ChipNeMo: Domain-Adapted LLMs for Chip Design
The first step in the design process is to generate a high-level design (HLD) from a set of requirements. This is typically done by a human designer, but it can be automated using Large Language Models (LLMs). ChipNeMo is a domain-adapted LLM specifically trained for chip design tasks. It can generate HLDs from natural language descriptions of chip requirements, significantly reducing the time and cost of the design process.

Robotic Factories Supercharge Industrial Digitalization as Electronic Makers Adopt NVIDIA AI and Omniverse
NVIDIA Omniverse, Isaac and Metropolis Enable Delta Electronics, Foxconn, Pegatron, Wistron to Digitally Build, Simulate and Operate Factory Digital Twins

两年一代，一年一款



agent自优化代码和算法是其自演进的必备能力

Coding是核心生产力，协助加速开发也占领生态心智

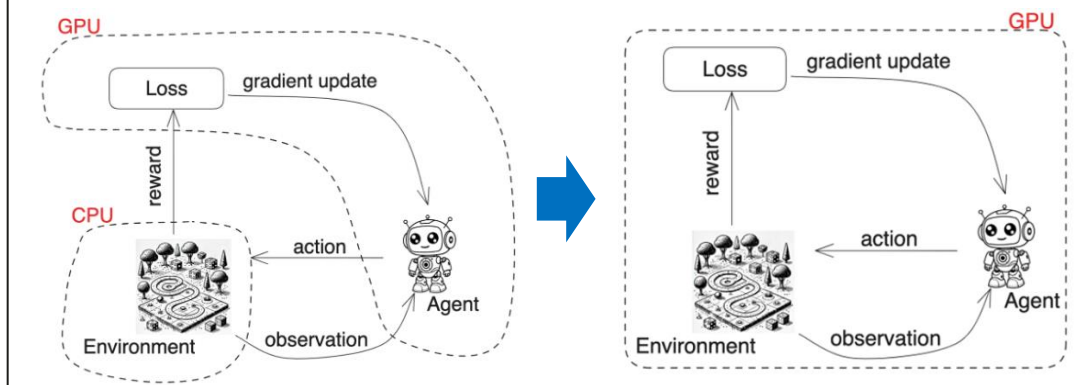
例子1: Gemini code assist (Gemini 2.0作为基模)-25/02/2025

和其他产品集成；在不需要人类反馈下，debug自己生成的代码

Gemini Code Assist uses large language models (LLMs) from Google. The LLMs are fine-tuned with billions of lines of open source code, security data, and Google Cloud documentation and sample code. These models paired with Gemini Code Assist give developers code completion, code generation, natural language chat, and more, in their IDE, and Google Cloud services including Firebase, Colab Enterprise (Vertex AI), Databases BigQuery, Apigee, and Application Integration.

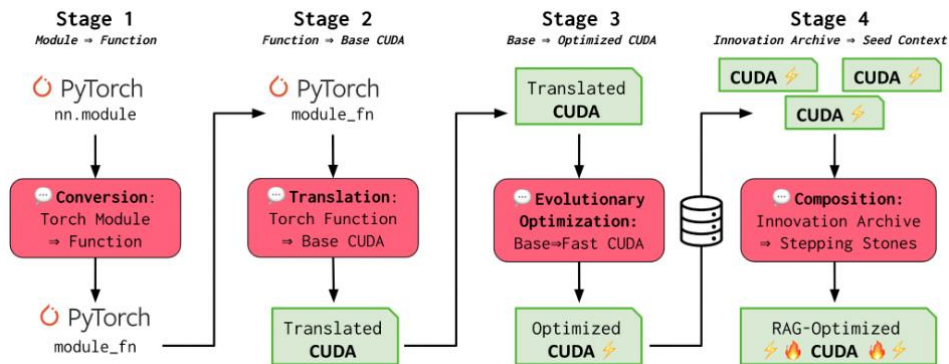
例子2: agent coding能力加速RL- Chris Lu, Jacob Forester

- 将原来跑在CPU的仿真环境迁移到GPU，实现RL加速1000倍
- 利用Jax，更容易地大规模部署在GPU；自动微分/并行等
- LLM用来代码迁移，自动生成和优化RL算法等



端到端CUDA工程师：250项CUDA任务中实现平均1.34倍加速

- 将模块torch代码转化为function,再转化为基线CUDA kernel
- LLM基于原有kernel优化，不断迭代
- 公开了30000由这个agent生成的CUDA kernel



Level 2 [Fused Ops]	# Translated Operations	Improved over torch	Improved over compile	Improved over KernelBench	Torch P50 Speedup	Torch P75 Speedup	Torch Max Speedup
deepseek-v3 [pass@10]	39	14	7	9	0.49	1.08	8.14
sonnet3.5 [pass@10]	52	20	14	13	0.39	1.27	10.81
deepseek-R1 [pass@10]	70	23	16	17	0.37	1.06	59.19
o1-preview [pass@10]	95	42	13	22	1.00	1.01	3.35
o1-high [pass@10]	81	29	17	19	0.87	1.00	4.34
deepseek-v3 [loop@10]	43	15	10	12	0.96	1.11	3.35
sonnet3.5 [loop@10]	64	17	15	11	0.31	1.03	10.16
deepseek-R1 [loop@10]	80	21	18	14	0.26	1.00	112.45
o1-preview [loop@10]	97	45	19	26	1.00	1.04	3.38
o1-high [loop@10]	90	24	13	15	0.51	1.00	6.78
o3-mini-high [loop@10]	88	20	20	17	0.23	0.93	8.64
Union - Max All	98	72	40	41	1.05	1.31	112.45

- 实现kernel优化，加速明显(发现验证过程有问题，这些数字不全可信)
- 但这个方法是可行
- 还不能理解底层硬件
- 有的kernel还慢了

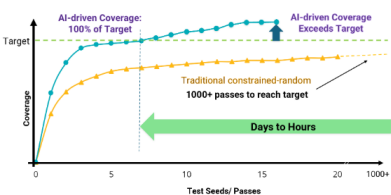
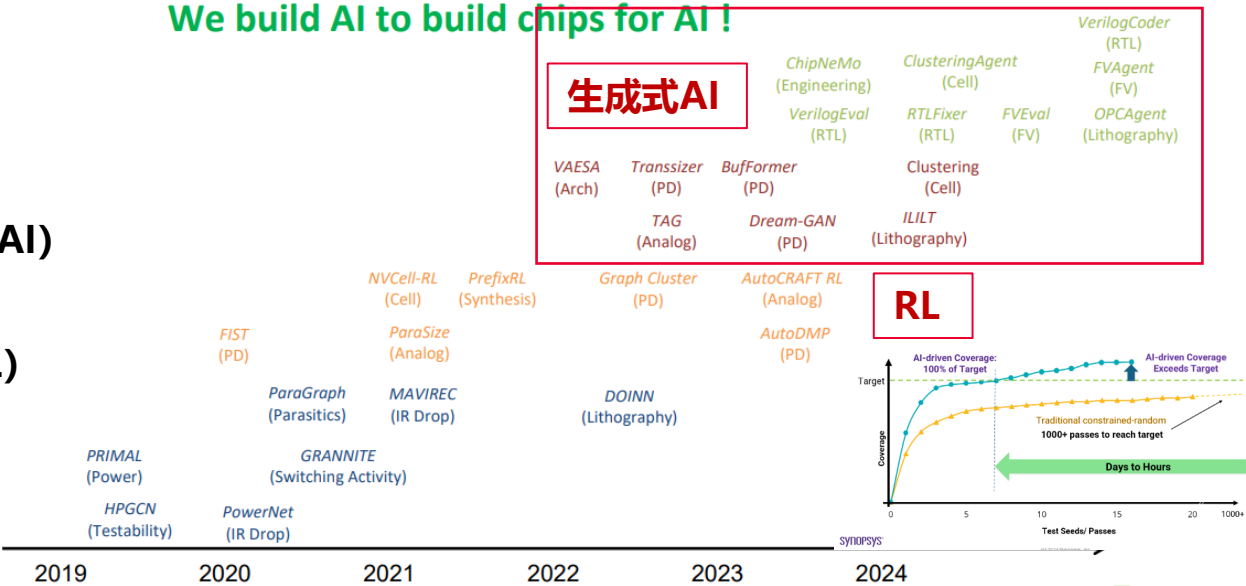
英伟达：用AI设计芯片来训练更好的AI

利用AI制造更好的芯片：更大范围使用AI，不断探索利用LLM的能力

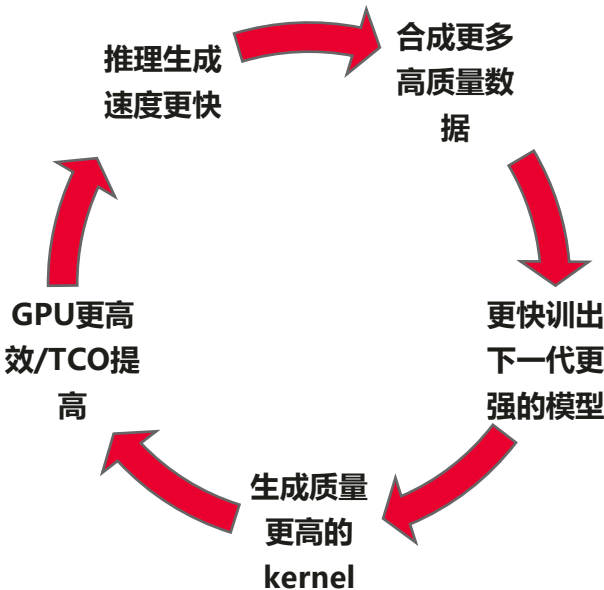
AI for Chip Design Research @ NVIDIA

We build AI to build chips for AI!

Design Assistance
设计辅助
Design Optimization
设计优化 (GenAI)
Design Optimization
设计优化 (RL)
Design Analysis
设计分析



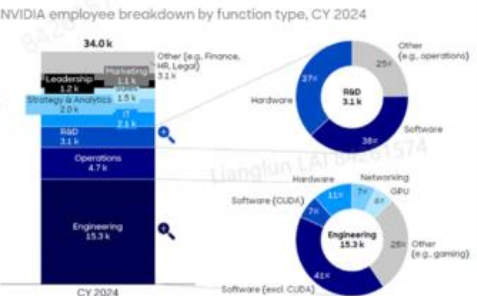
加速飞轮：Kernel优化加速inference scaling



英伟达软件工程师占比高，LLM增强coding和CUDA kernel优化效益高

Side note:

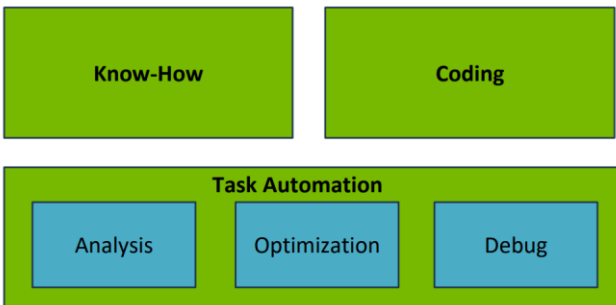
- 工程师约占员工总数的45%，其中约48%专注于软件开发，包括约7%专门从事CUDA相关工作
- 研发部门硬件和软件各占约37%和38%



在设计芯片时，工程师通常面临众多选择和约束。

设计空间包括逻辑、面积、单元库、工艺参数、电子设计自动化（EDA）工具选择及配置之间的权衡。AI可用于探索极其庞大的设计空间，能够协助设计者探索设计空间，并整合领域特定的知识。

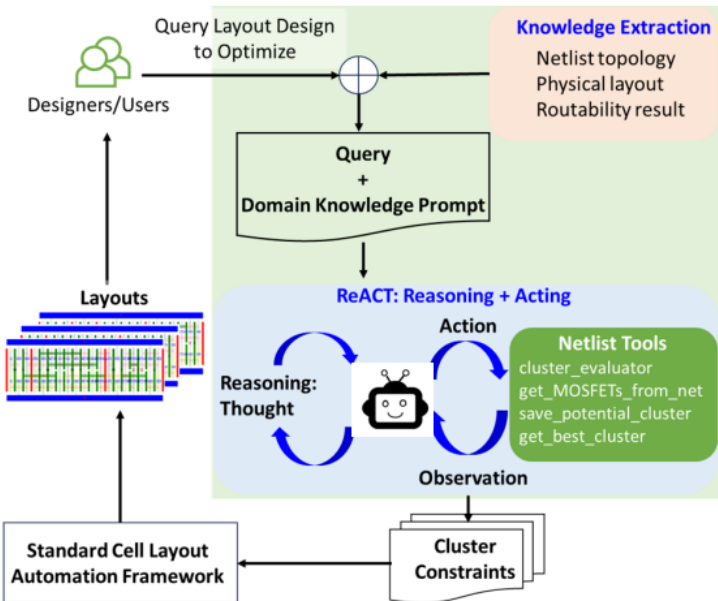
LLM设计辅助：经验+coding+任务自动化



设计优化：结合LLM的语言和推理能力和领域知识(设计认知以及特殊工具)，构建了芯片设计agent, 有效提高设计性能和效率

用agent减少cell placement 5%的面积开销

- 利用LLM的自然语言和推理能力，逐步生成高质量的cluster约束，以优化单元布局的PPA，并使用ReAct调试routability。
- 在2纳米的时序标准单元评测中，不仅成功地将减少cell面积，还能够修复cell布局设计中的routability问题。



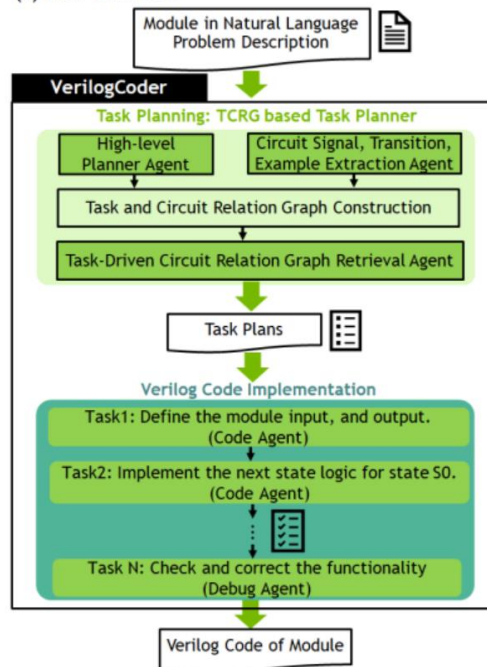
使用预先设计和验证的逻辑单元来构建复杂的电路

C.-T. Ho et al, Large Language Model (LLM) for Standard Cell Layout Design Optimization

VerilogCoder Agent生成正确的Verilog代码正确率94.2%，比SOTA提高33.9%

- 构建了生成Verilog code的AI Agent系统
- 自动调用Verilog工具 (i.e., syntax checker, simulator, and waveform tracer)
- 两级结构，先通过多LLM agent协同+retrival作为任务规划；再通过coding+debug agent执行子任务验证

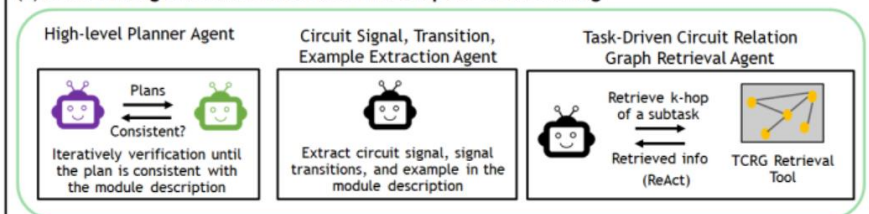
(a) Flow Overview



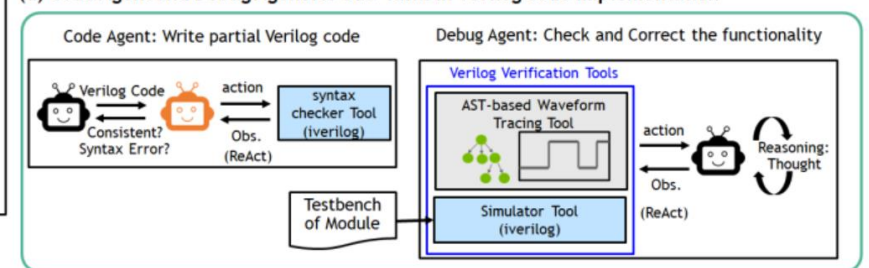
(b) Large Language Model (LLM) Roles of Multi-LLM Agents in VerilogCoder



(c) Multi-LLM Agents with Various Roles for Steps in Task Planning



(d) Code Agent and Debug Agent for Sub-Tasks in Verilog Code Implementation



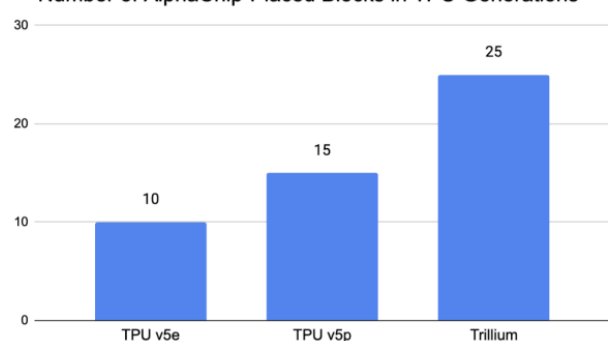
Verilog HDL（简称Verilog）是一种硬件描述语言，用于数字电路的系统设计。可对算法级、门级、开关级等多种抽象设计层次进行建模

C.-T. Ho et al, VerilogCoder: Autonomous Verilog Coding Agents with Graph-based Planning and Abstract Syntax Tree (AST)-based Waveform Tracing Tool

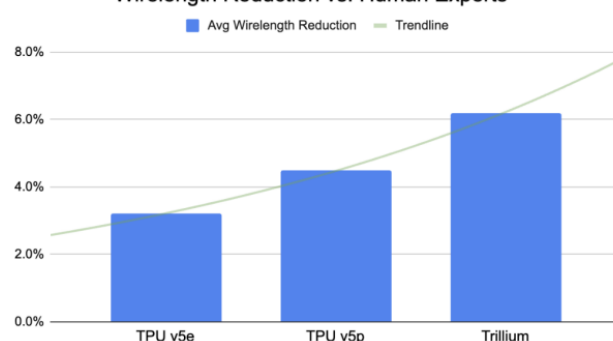
谷歌：AI加速算法，软件栈，硬件的迭代

AlphaChip已被部署在三代以上的TPU中。在每一代中，它在更多的模块中被采用，并且以更大的优势超越了人类专家。

Number of AlphaChip-Placed Blocks in TPU Generations

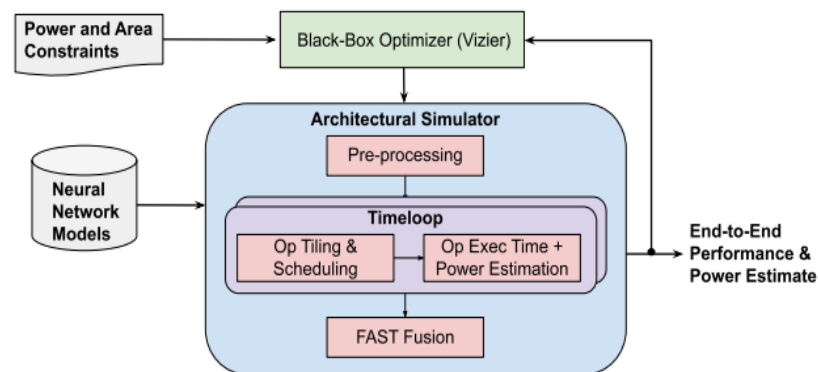


Wirelength Reduction vs. Human Experts



Mar 2024: 7 AlphaChip layouts adopted in Google Axion Processors (ARM-based CPU).

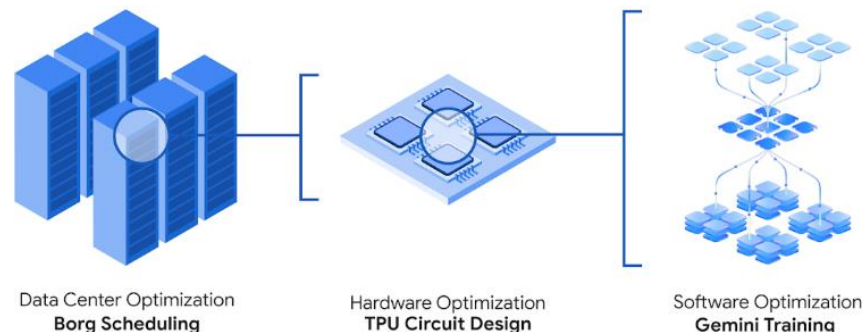
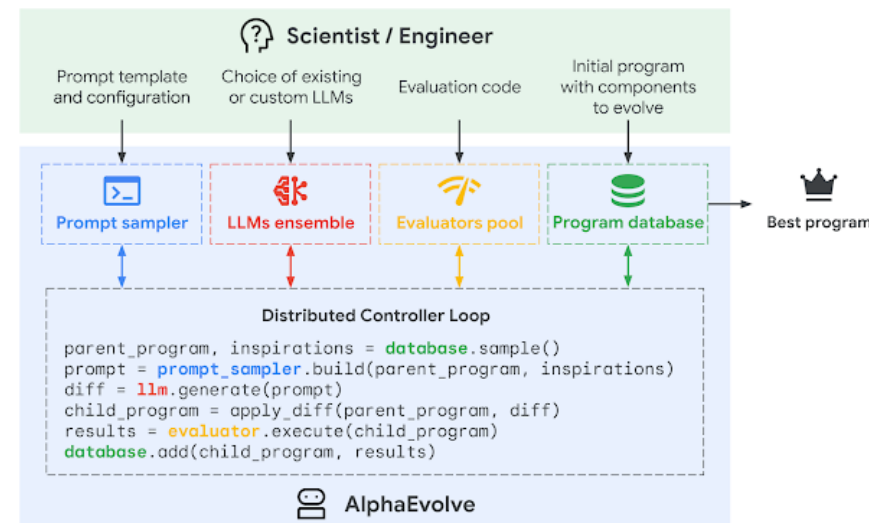
Alphachip+全栈加速器搜索技术: 利用强化学习平均提高Perf/TDP3.7倍



· **Jeff Dean: AlphaChip是实际生产验证过有效的:** “build enough trust for the TPU team to use our layouts; deliver TPU chips and make them as efficient and reliable as possible, and they cannot afford to take unnecessary risks.”

全栈加速器搜索技术, 涵盖了硬件-软件堆栈中的关键设计决策, 包括硬件数据路径、软件调度及编译器等

AlphaEvolve:数据中心调度, 硬件设计, AI训练和推理, 数学和算法发现



公布时间的间隔缩短, 从3年到1年: 2018 V3 -> 2021 V4 -> 2023 V5 -> 2024 V6; 预计2026 V7量产

谷歌AI co-scientist科学智能自主探索：利用TTC scaling持续self-play，激发模型提出自我创新

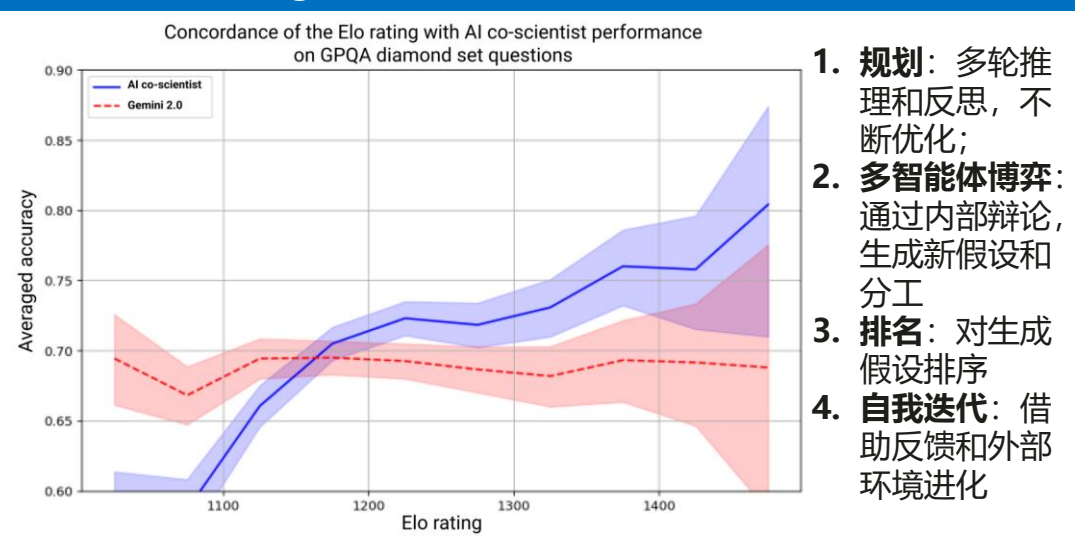
co-scientist基于Gemini 2.0 multi-agent架构，并采用了异步任务执行框架

- **异步任务框架[异步补时延]**：agent在异步、连续和可配置的任务执行框架内作为工作进程运行。一个监督agent管理工作任务队列，将这些进程分配给子任务agent，并分配资源。这种设计使系统能够灵活有效地利用计算资源，并迭代提高其科学推理能力。
- **自然语言界面**：通过自然语言与系统交互并进行监督。定义初始研究目标，对生成的假设（包括解决方案）提供反馈，并总体上指导系统的进展。
- **专业Agent**：根据归纳偏见和科学先验，分解子任务。每个专门的agent都配备了定制的指令提示。
- **上下文记忆**：在长时间范围内实现迭代计算和科学推理，使用持久上下文记忆来存储和检索计算过程中agent和系统的状态。

AI Co scientist多agent系统：在人类给出的研究目标后，连续生成，评估，讨论和提高研究的假设和建议



利用TTC scaling循环增强：通过循环，连续自我演进；利用了模型的reasoning和反思能力



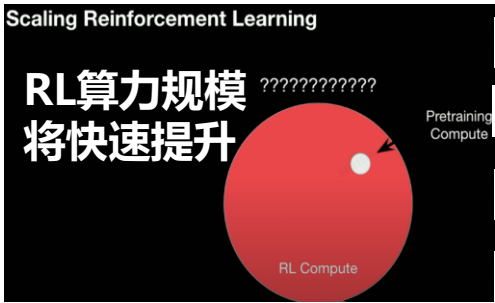
Cong Lu观点: 1. 人处理信息，交互信息的能力是有限的，agent可以scaling；2. open ended任务生成，用agent设计越来越复杂的任务，从而模型可以进化；3. 利用更大的context，Agent有效利用失败的尝试，学会反思；4. 发现并运用新的loss function；5. chains of agents，即是利用了LLM的知识库混合概念，也利用不同agent的特点协作。

Agent算力底座

Agentic RL训练：下一代模型的开放环境，长轨学习和自进化都依赖Agentic RL Infra

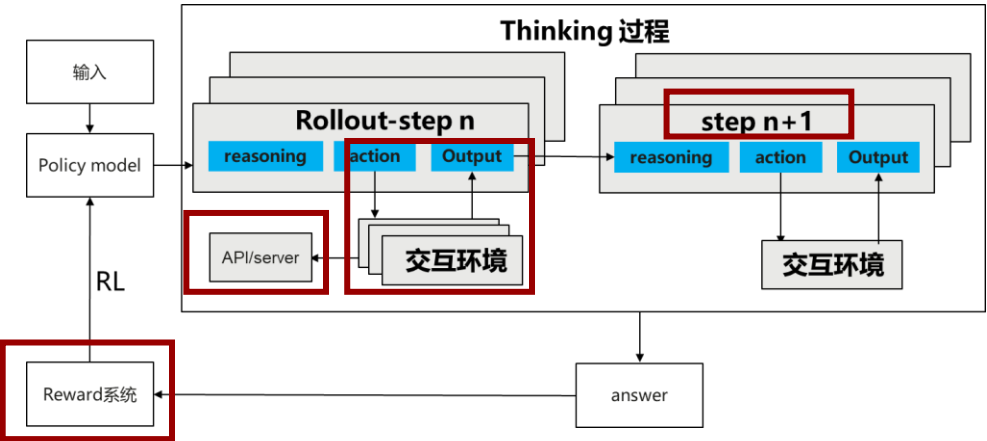
- RL训练随agentic tool use变得复杂，rollout生成的时间更长，长尾差异更大；且RL总体算力加速提升，进一步提高计算资源利用率成为刚需

Agentic RL是训练下一代模型的前提，且RL规模会进一步scaling



- 环境探索Scaling
- 轨迹Scaling
- Agents Scaling
- 自进化 Scaling

Agentic RL: 新增多模态，多环境，多工具，多轮次，多agent，多reward的特点

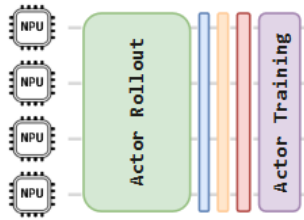


例子：字节Retool；微软agentic tool use；Nemotron-Research-Tool-N1

挑战1：多模态RL生成瓶颈更严重==集群所需算力剧增

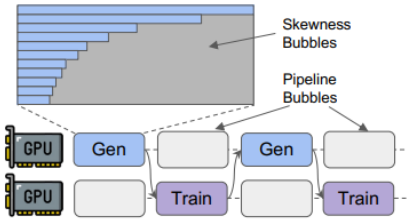
	LLM (32B)	多模态理解 (3B)	多模态生成 (2.7B)
样本生成耗时 (s)	360.10	87.66	383.05
训练耗时 (s)	293.20	93.12	31.85
样本生成耗时占比	55.12%	48.49%	92.32%

多模态生成样本生成时间长，需要支持10步以上的异步



挑战2：生成-环境交互-reward-训练所需资源差异大，大规模多轮训练下共卡部署无优势

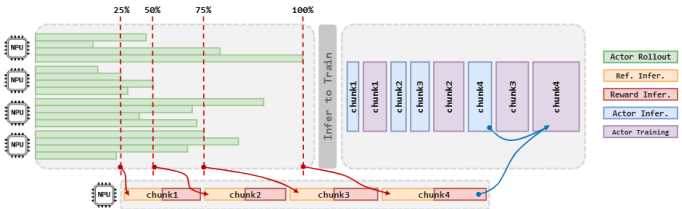
挑战3：引入工具调用后，多轮环境交互使得长尾空泡，pipeline空泡更严重==集群利用率大幅降低



挑战4：reward不再是单轮后的单次前向计算；多轮、多层级、多样奖励拉长reward阶段和其复杂度==调度难度增加

S0	c	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13
Hidden State	Task Context	Agent output				Env. Output				Agent Output				
Per-token PPO														
Per-turn PPO														
Per-trajectory PPO														

挑战5：探索更多异步步数，当前四野支持4步准异步，异步步数对精度影响仍不明确

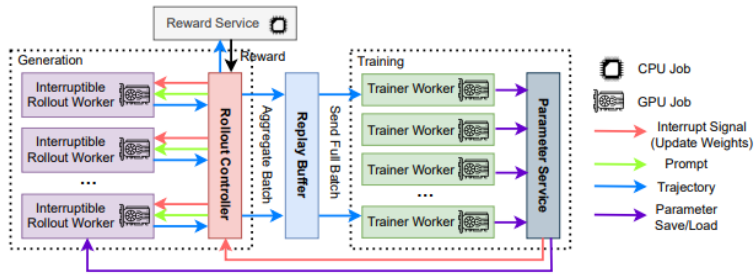


吞吐和效率优化思路：从系统，算法和编排提高RL系统端到端吞吐和算力资源利用率

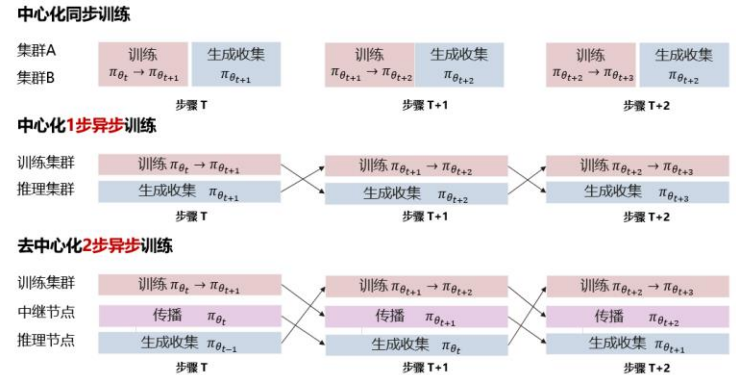
- Agentic RL infra需新增多个特性，业界25H2将加速研发，建议立即准备

推-交互-训分离，异步RL实现多倍加速 25.06蚂蚁AREAL-实现2.77倍加速

全异步；控制数据陈旧度；动态微批次算法；可扩展CPU部署支持复杂reward service



25.04 Intellect-2分布式异步RL



支持异步的RL算法减少精度损失 25.06 Meta LlamaRL: 405B policy实现10.7倍加速

面向异步训练的针对优化RL算法AIPO - Asynchronous Importance weighted Policy Optimization, 减少off policy对训练精度的影响

$$\sum_{t=1}^T \min \left(\frac{\pi(y_t | x, y_{1:t-1})}{\mu(y_t | x, y_{1:t-1})}, \rho \right) \cdot A(x, y_{1:t}) \nabla \log \pi(y_t | x, y_{1:t-1})$$

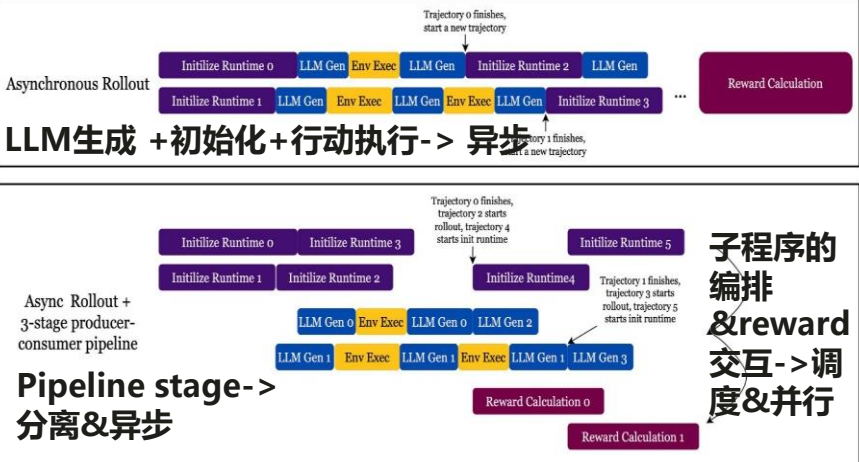
25.06蚂蚁AREAL

- 数据陈旧度感知训练
- 支持policy版本不一致
- 通过解耦PPO对一个batch训练数据要来自同一个policy的要求，用 π_{behav} 等效不同版本policy

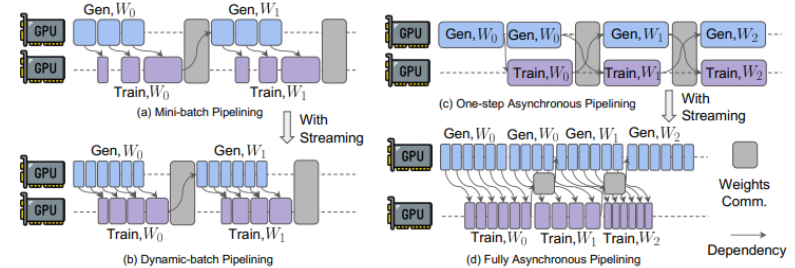
$$J(\theta) = \mathbb{E}_{q \sim \mathcal{D}, a_t \sim \pi_{\text{behav}}} \left[\sum_{t=1}^H \min \left(\frac{\pi_{\theta}}{\pi_{\text{behav}}}, \hat{A}_t, \frac{\pi_{\text{prox}}}{\pi_{\text{behav}}} \text{clip} \left(\frac{\pi_{\theta}}{\pi_{\text{prox}}}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]$$
$$= \mathbb{E}_{q \sim \mathcal{D}, a_t \sim \pi_{\text{behav}}} \left[\sum_{t=1}^H \frac{\pi_{\text{prox}}}{\pi_{\text{behav}}} \min \left(u_t^{\text{prox}}(\theta) \hat{A}_t, \text{clip} \left(u_t^{\text{prox}}(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]$$

高效流水编排和并行，提高算力利用率 25.05 SkyRL: 多阶段异步并行

Agentic RL训练环境交互时间存在不确定性，多层次多步异步提供更高的灵活性



StreamRL: 动态微批次流水和异步减少空泡



Agent长任务优化：环境计算将成为训练和部署阶段的关键一环

- 做好长轨迹RL和环境计算将支持agent应用到更多的真实业务场景

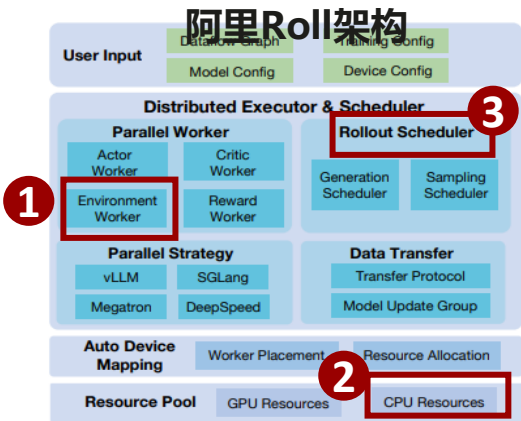
RL架构需针对长任务的环境计算升级

挑战1：环境资源消耗巨大：如 SWE-Bench每个容器至少需要>1 CPU 和约 7GB 的存储。批量大小为 16, 8 个并发rollout可能需要超过 100 个 CPU 和接近 1TB 的磁盘空间。

挑战2：CPU运行环境和工具慢，阻塞GPU，拉低系统效率

挑战3：环境worker异步并行；rollout scheduler-突破传统批量处理的限制，以单样本粒度管理，并支持优先级样本调度

挑战4：系统稳定，快速恢复：当agent需要持续数小时或数天工作，环境计算需保持稳定和生成结果的鲁棒性

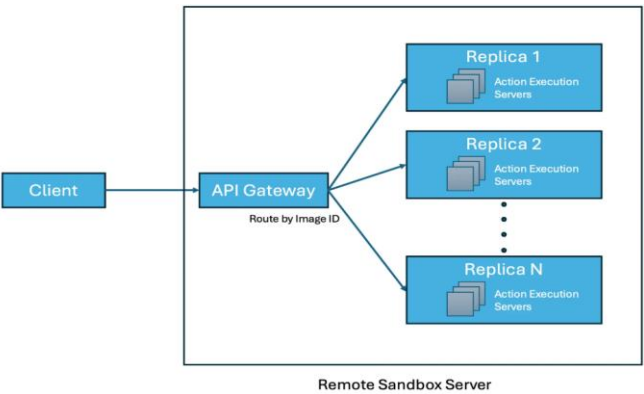


agent需要隔离且有状态的环境

OpenAI Codex: 真实的学习环境：

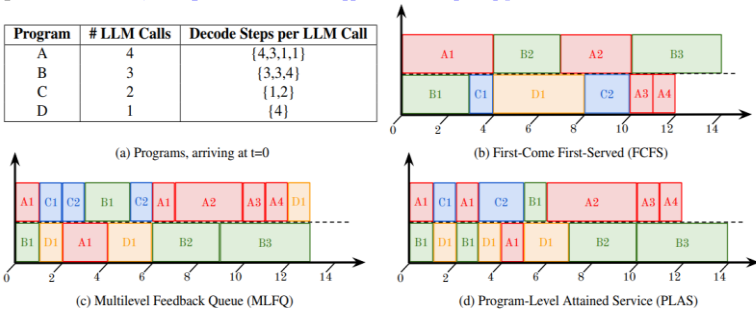
- 训练与生产部署使用相同环境和相同的容器化基础设施
- 构建更小、测试的更好的模块

SkyRL: 可扩展的环境worker：利用远程沙盒服务器，将环境执行与训练过程分离，使环境可以独立扩展，从而维持高频率的环境交互，确保在rollout阶段GPU资源得到高效利用。



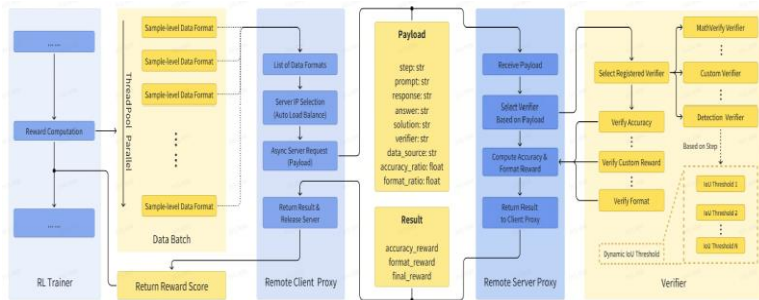
编排提升环境交互效率

包含多个子任务，子任务rollout长短和数量不同：**采用不同的编排优化策略**



25.05Minimax Vriune异步reward

RL trainer和reward服务器里不同类型的验证器交互，根据子任务异步返回多样reward



Agent推理总体趋势：算力变大，内存变高，时延变长，调度弹性

	reasoning模型	Agentic模型 ~长任务
算力	Test time scaling-单轮算力	多轮高算力；多模型部署；多工具部署
内存	单轮次cache	多轮次长序列，stateful, 任务质量依赖长记忆
互联	部署模型本身流量	agent通信协议MCP, A2A流量；机-机交互+机-环境交互带来新的流量特征
调度	部署模型的调度	动态调用工具/应用; 多agent调度；高时延响应下保证SLO；

需要面向agent设计推理系统，也利用agent感知系统资源，更高效管理，并自我优化

算力大：推理token开销15X

Anthropic博客：单agent和多agent系统的token消耗量分别比chat多4倍和15倍

tokens fast. In our data, agents typically use about 4× more tokens than chat interactions, and multi-agent systems use about 15× more tokens than chats. For economic viability, multi-agent systems

变化：推理需求未来可增长1000x；需要更大规模推理集群

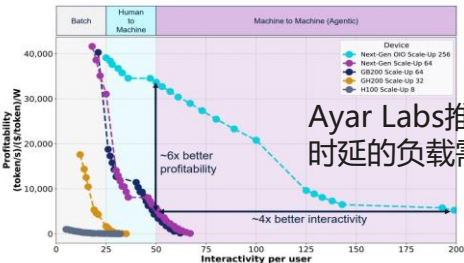
内存高：多轮次的长序列增大KV cache

- Agent原生记忆和stateful工具都有memory开销
- Agent完成长任务的性能依赖长记忆，交互时间长，需要高效管理cache，利用多级存储
- Agent个性化意味着记忆开销不断增长；叠加多agent，memory可能超线性增长

变化：对内存的需求将不断增长; 需要池化memory

时延长：工具调用使时延变长

- 快慢思考和工具调用结合使用，增加端到端时延
- 随着agent独立完成任务时长增加，协议MCP, A2A流量可能剧增：机-机交互+机-世界交互比人-机交互频率更高，时延更低

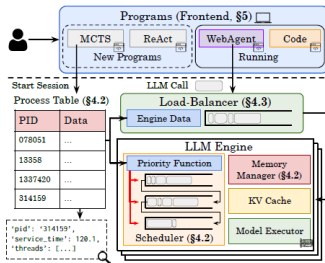


Ayar Labs推断agent高并发低时延的负载需要光I/O支持

变化：机-机交互低时延和工具调用时延变长冲突

调度弹性：任务动态编排和推理高可靠

- 能支持长任务下不同子任务的优先级调度
- 推理系统稳定，可快速恢复：agent长时任务需生成结果鲁棒
- Agent任务差异大，所需算力资源变化大，需弹性扩缩



变化：需动态编排模型/工具和弹性调度算力资源保证系统高效率

未来猜测：自主agent需要需要新的软硬协同，提升Agent Flops Utilisation (AFU)?

问题：自主Agent未能充分利用给定的资源

问题1：计划小模型的时候用CPU, 大模型用GPU

问题2：OAI尝试的三个不同的agents的解题策略都不能高效的利用提供的算力和允许的时间

问题3: 有时候会超过内存或存储, 导致任务提前中断

多给一个GPU, 同样时间内没有提升解题效果

Table 3: Comparing the performance of GPT-4o (AIDE) on different hardware configurations (averaged over 3 seeds, except for *Standard* which used 36 seeds).

Setup	Description	Achieved Medal (%)
CPU-only	Same as <i>Standard</i> but with no GPU provided.	9.1 ± 1.0
Standard	36 vCPUs with 440GB RAM and one 24GB A10 GPU.	8.7 ± 0.5
Extra GPU	Same as <i>Standard</i> but with two 24GB A10 GPUs.	10.2 ± 2.0

agent几乎没有自我思考它们的代码会跑多久

1. Agent/模型不知道pass一次所需的时间和算力
2. 不能理解系统提供的算力
3. 也就无法充分利用系统所有的计算资源

潜在原因1：训练时没有‘思考如何利用算力；代码怎么跑的更高效；代码会需要多少资源；’这一类问题内在思考的训练

潜在原因2：现有的推理系统还是面向开发人员，需要人去设计配置局部最优解

为什么关心这个问题？=Agent加速AI开发

- 从Coding agent到大模型研究员agent
- 自优化/自演进agent

模型资源利用率高，不代表agent利用率高

- 有/无人工介入下，agent的资源利用率
- **Agent Flops Utilisation (AFU)**
- **Agent bandwidth Utilisation (ABU)**
- **Agent memory Utilisation (AMU)**
- 如何异构算力CPU/GPU；什么组合合适？A+K

面向Agent的推理系统设计

模型+agent推理框架需要能够

1. 感知底层的算力资源
2. 感知自身及任务的时间和算力开销
3. 制定利用算力资源的策略
4. 感知资源利用率并自我优化

2025-2026: Agentic能力评测定义agent能力边界，牵引模型迭代

Why?: 评测定义模型优化目标, 是前沿探索的驱动力和指引, 通过其筛选合适训练数据, 定义模型能力边界, 引导模型迭代

Deepmind- Zhengdong Wang

1. 模型实验需要一个足够明确、足够快速、并且足够接近所定义的“更好”的测试。
2. **AI研究者唯一在做AI研究的时候就是他们选择评估标准的时候。**
3. 评测是指导AI开发非常重要的部分, 现有评测随不完美, 但清晰且快速。
4. 适合AI解决的问题的三个特性: 一个巨大的组合搜索空间、一个明确的优化目标函数, 以及大量的数据或一个高效的模拟器
5. 有时候无法确定模型该怎么表现或者定义指标, 这时候需要**用交互定义模型能力边界**

OAI-Jason Wei

如何构建模型的评测?

- 提高评测得分是研究的驱动力, 知道模型取得突破是因为评测得分大幅提高
 - 关键任务是定义要优化什么评测指标
 - **成功的评测集和技术关键突破常常是一体的**
- ### 好评测的特征:
1. 需要足够多的样品, 这样在模型开发过程中, 不会有噪音信号, 表现能够稳定;
 2. 高质量eval, 很少的错误, 评测的ground truth要足够solid, 尤其对于超强模型来说
 3. 容易使用, 结果容易理解和呈现
 4. 容易在Infra上跑起来
 5. 评测测试的东西或者能力本身要有现实意义,
 6. 打分要无比正确
 7. 评测要足够难, 要一段时间都不会饱和

MiniMax闫俊杰

1. 技术就是要不断提升上限, 这需要定义好下一代能力。最大的驱动力是技术进化, 这需要算力, 数据, 钱, 和足够好的人。
2. **非常清晰低定义模型能力分级, 然后不断提升, 需要什么样的算法, 数据和推理过程, 技术手段去逼近指标;**
3. 头部公司有大量内部定义的benchmark, 一些自己的底层思考和设计
4. 模型不是基于功耗反馈和数据迭代采编好的, 关键是先定了一个技术benchmark, 才做到的;
5. 思考先优化什么benchmark, 基于这个领域是否够收敛, 以及能创造多大的独特价值
6. 不同任务怎么用benchmarks来衡量, 即使是o3, 它在一些评测的分数也很低

启示:

1. Benchmark构建的本身是对要解决问题、目标场景, 训练数据的深入理解; 因此构建一个好的测试集将牵引模型前沿能力开发
2. 测试集是反映所用技术手段好坏的依据, 如果测试集不准就有可能选择错误的技术
3. 随着模型加速迭代, 能自我改进, 更高效生成高质量benchmark将是竞争力之一

What: Benchmark: 真实场景, 准确性, 鲁棒性, 高价值的长任务和推理性价比

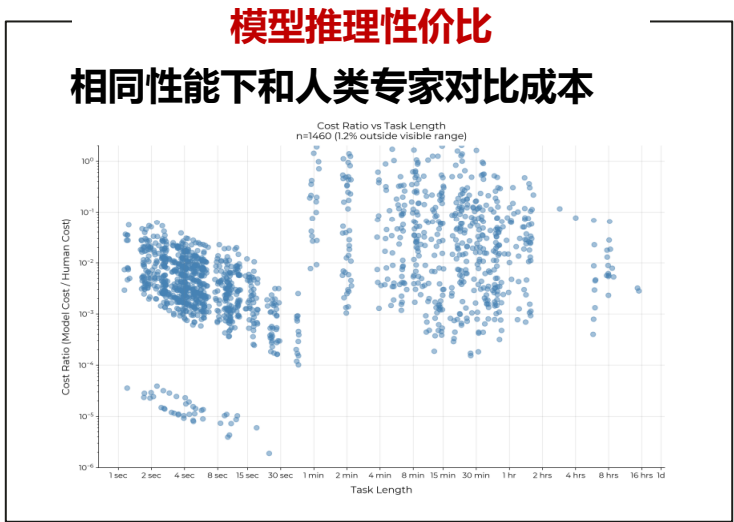
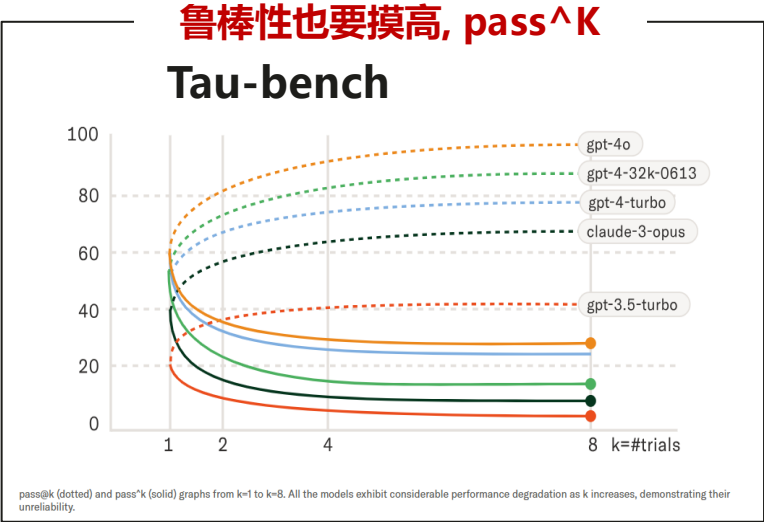
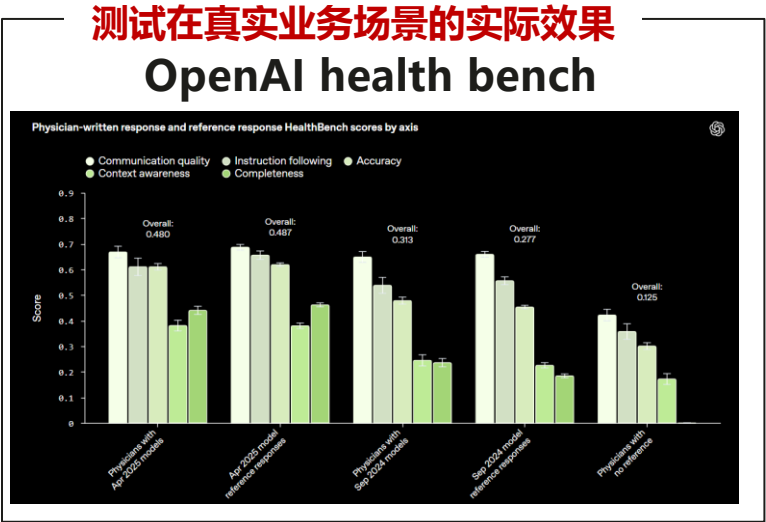
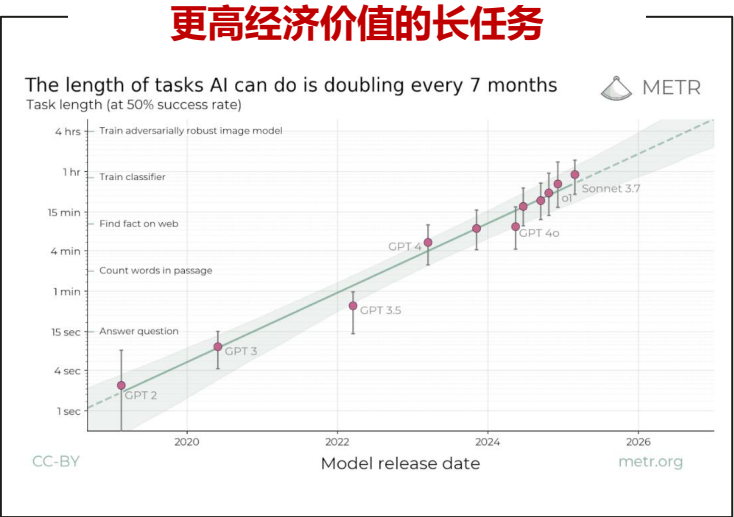
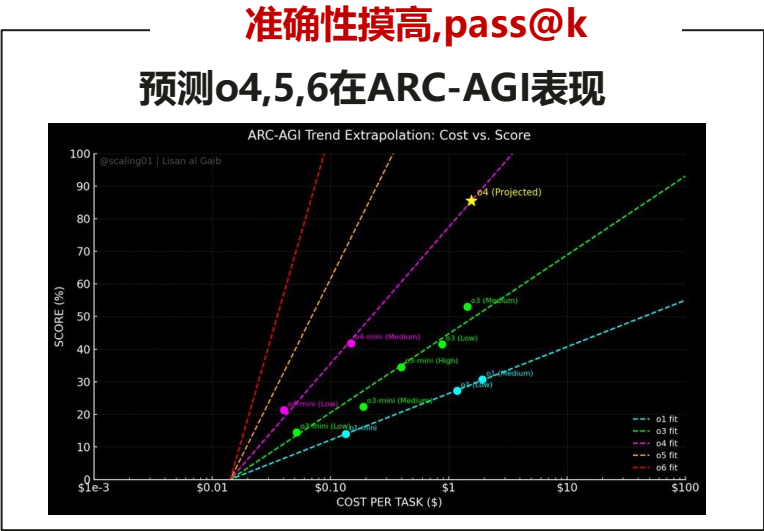
Agent体验

任务复杂度~耗时

完成效率~价值\$-成本\$

完成准确性~pass@k;

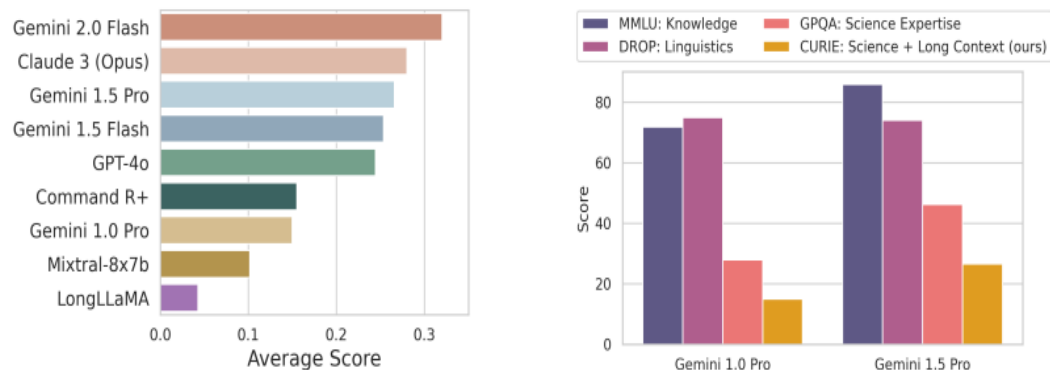
完成鲁棒性~pass^k



复杂度：现有模型在长期任务（自动软件编写和科学难题）都不足

谷歌Curie benchmark: 衡量实现长期目标的能力

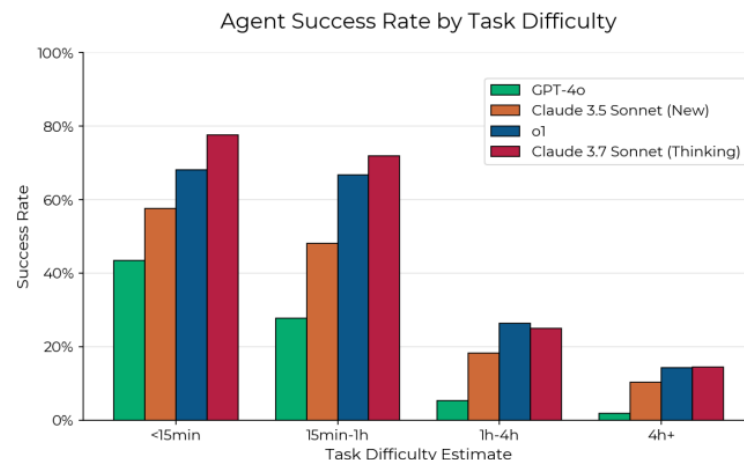
- **测试模型在科学领域的的能力，解决现实科学难题**
- 具体包括模型理解图像，理解表格，以及实验当中工具使用的原因，如何选择合适的工具等
- 数据集的构建非常昂贵：首先依赖人类专家定义指标，并从现实的任务中改写成模型可以解决的子任务
- 依据任务难度分级并制定可以测量模型能力的指标



Gemini得分率不足40%

METR-HCAST: 测试自动编写软件的能力

<1h的任务成功率 70-80%， >4h的任务成功率 20%



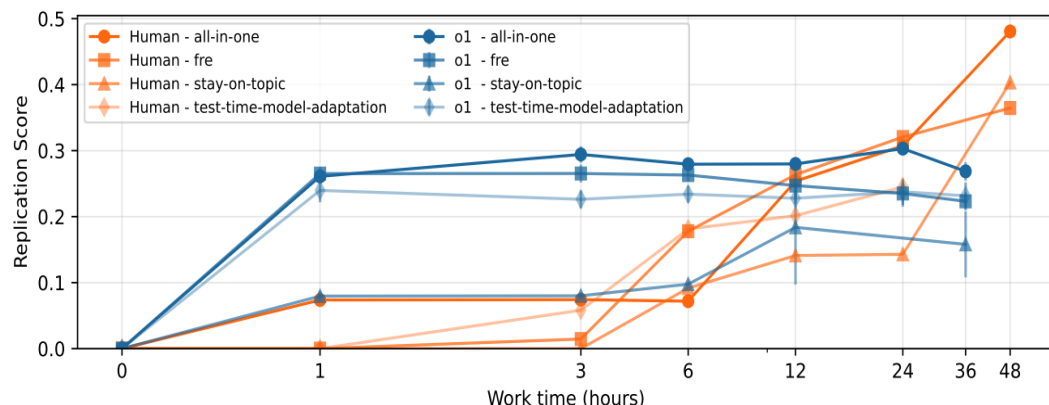
不足：多数任务都是<10步；测试集大多是15min以内的短任务



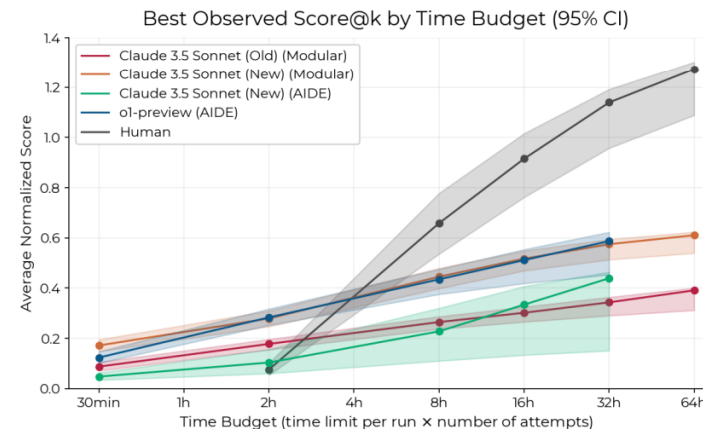
准确性：模型完成AI研发相关任务的能力暂时还不如人类专家

Paperbench：测试模型完成现实ML R&D的能力

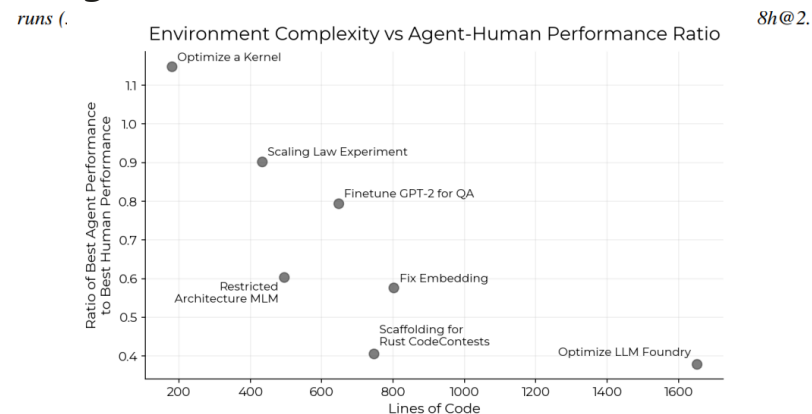
- o1的得分随时间变化：在第一个小时快速上升，然后就几乎不变，难以继续提高【模型有强大基础知识，能在一开始写大量代码，但是无法继续提升】
- 人类得分随时间变化：一开始几个小时得分上升速度慢，但能够持续提升
- 数据集创建：需要人类专家花费数天构建，需要深入理解任务，分解任务，设立目标
- 测试，监控和预测AI系统完成现实ML研发工作的能力，将对AI自动化完成ML研究起促进作用
- 【复杂任务的评估做分解，从子任务层级做评估，其实还是outcome based】
- 子任务之间是有依赖关系的
- 评估的judge模型越强，任务分解的颗粒度越小



METR-RE-Bench：测试模型完成AI R&D任务的能力



Agent能力和人类专家的对比，表现提升变缓

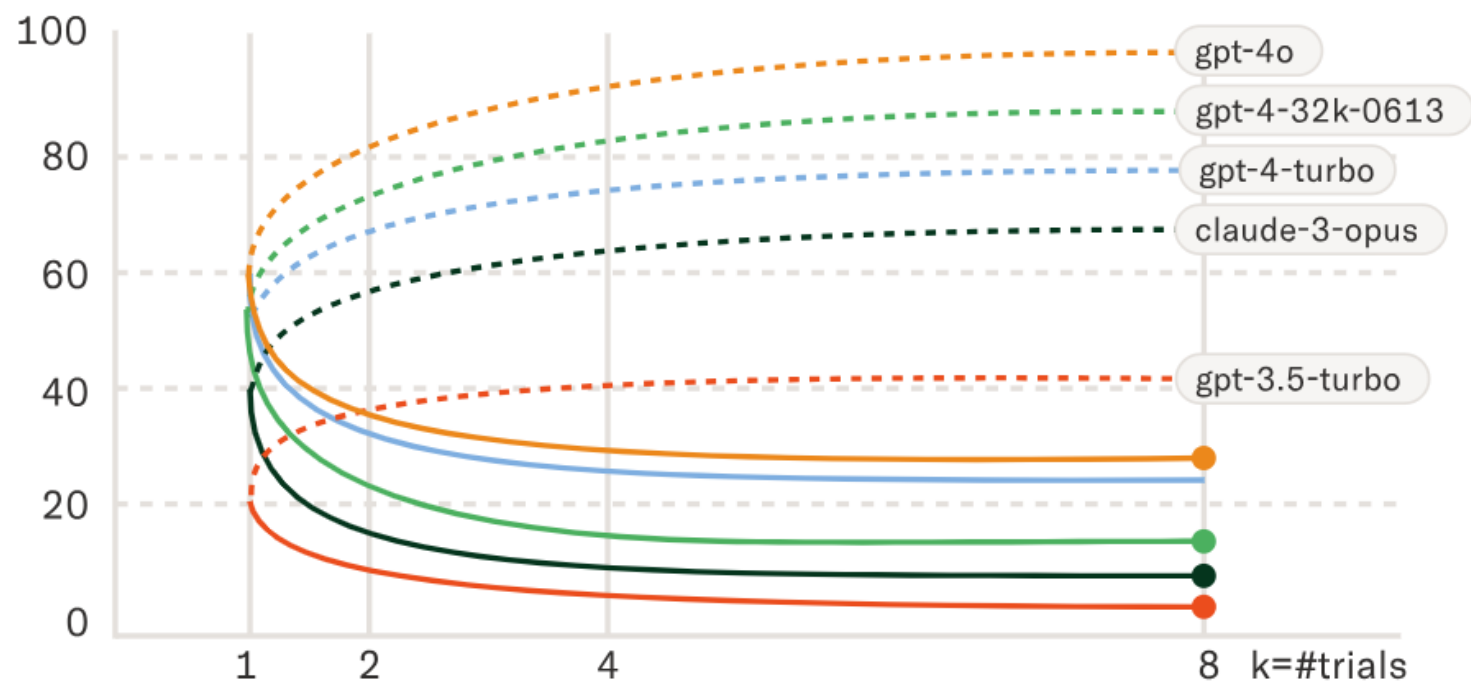


越复杂的任务，agent和人类专家的差距越大

启示：1. 测试模型复现实验的能力，模型可以用来快速复现其他实验室的成果，然后复制成功，大幅加快模型迭代周期

鲁棒性：Tau-bench，现阶段agent很弱

$$\text{pass}^k = \mathbb{E}_{\text{task}} \left[\frac{\binom{c}{k}}{\binom{n}{k}} \right], \quad \text{pass}@k = 1 - \mathbb{E}_{\text{task}} \left[\frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$



pass@k (dotted) and pass^k (solid) graphs from k=1 to k=8. All the models exhibit considerable performance degradation as k increases, demonstrating their unreliability.

- Agents能力三个要求【1. 能够长时间和人类交互，获得更多信息并理解意图；2. 准确的遵循任务的失灵和规则，完成某项任务；3. 在大量的交互中保证其一致性和鲁邦性】
- 现阶段还是agent很弱的：：
agents built on top of LM
function calling lack sufficient consistency and rule-following ability to reliably build real-world applications.
- 需要提高agents long-horizon information tracking and memory 和the ability to focus on the right pieces of information in context for the decision at hand, especially when there may be conflicting facts present.

建议布局agentic评测：How? 真实，准确，可扩展，全面

趋势1：数据来自互联网 -> 真实应用场景

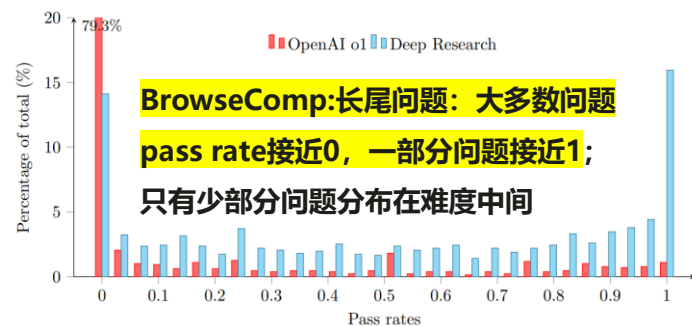
选择反应真实世界agentic行为的任务

- SWE-Bench: 解决真实软件工程问题
- KernelBench: 加速kernel的能力
- ToolEval: 评估使用工具能力的自动评估器
- MLE-Bench: 设计，建设训练ML模型的能力
- BrowseCom: AI网络浏览能力
- ARA: 模型获得真实世界更多资源的能力
- OpenAI PRs: 自动做AI研究的能力
- Agent flops 利用率: 感知算力资源利用率

挑战：如何在实操环境搜集任务和ground truth

趋势3：离线 -> 自动动态更新

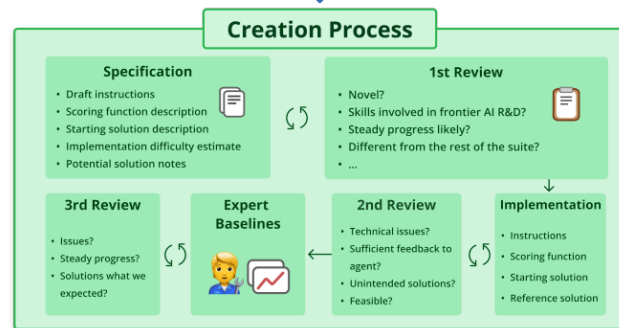
- 难度分级并自动更新：测试基础和进阶能力
- 测试相关性：添加新挑战测试新能力
- 减少测试污染



挑战：如何高效可扩展

趋势2：专家预先编写 -> 交互构建任务

METR: 目前测试集需要人类专家花费数天构建，需要深入理解任务，分解任务，设立目标



挑战：如何利用agent的交互记录

趋势4：人类先验bias -> 更真实

- AAEF评测框架和多维度的复合评测：工具使用效率，记忆一致性，规划，端到端实现

$$TUE = \alpha * (\text{Tool Selection Accuracy}) + \beta * (\text{Tool Usage Efficiency}) + \gamma * (\text{API Call Precision})$$

$$MCR = (\text{Context Preservation Score} * \text{Information Retention Rate}) / (1 + \text{Retrieval Latency})$$

$$SPI = (\text{Goal Decomposition Efficiency} * \text{Plan Adaptability}) * (1 - \text{Plan Execution Error Rate})$$

$$CSS = (\text{Cross-Component Utilisation Rate} * \text{Workflow Cohesion Index}) / (1 + \text{Component Conflict Rate})$$

- HealthBench: 不同专家有评价的差异不同的原则，模型打分如何可信？

挑战：如何超人类去验证准确性

启示和建议

1. 组织专业领域/工作流的专家构建评测集：不同背景，不同专业特长；对专家有严格的筛选(资格，投入，质量等)
2. 针对agent应用特有的长任务测试，调用真实工具测试；开放，动态，真实的场景问题，而不是简单的QA或多项选择题
3. 多颗粒度Rubric评测法：依靠人类专家先验和模型，编写一些列评分标准，模型的回答会根据每个问答特定的评分标准进行打分
4. 更新测试集难度，frontier模型的正确率要不足50%

agent协议

MCP应对MxN难题，定义模型-应用间的交互和通信，已成业界标准

Anthropic定义：统一的标准化协议管理LLM与外部环境的交互

MCP (Model Context Protocol, 模型上下文协议) 是一种新标准, 用于将人工智能助手连接到数据所在的系统, 包括内容存储库、业务工具和开发环境。其目的是帮助前沿模型产生更好、更相关的响应。

随着人工智能助手获得广泛应用, 业界在模型功能上投入了大量资金, 在推理和质量方面实现了快速进步。然而, 即使是最复杂的模型也会受到与数据隔离的限制——陷入信息孤岛。每个新数据源都需要自定义集成, 这使得互联系统难以扩大规模。

MCP 解决了这一挑战。它提供了一个通用的开放标准, 用于连接人工智能系统与数据源, 用单一协议取代分散的集成。结果是一种更简单、更可靠的方式让人工智能系统能够访问它们所需的数据。

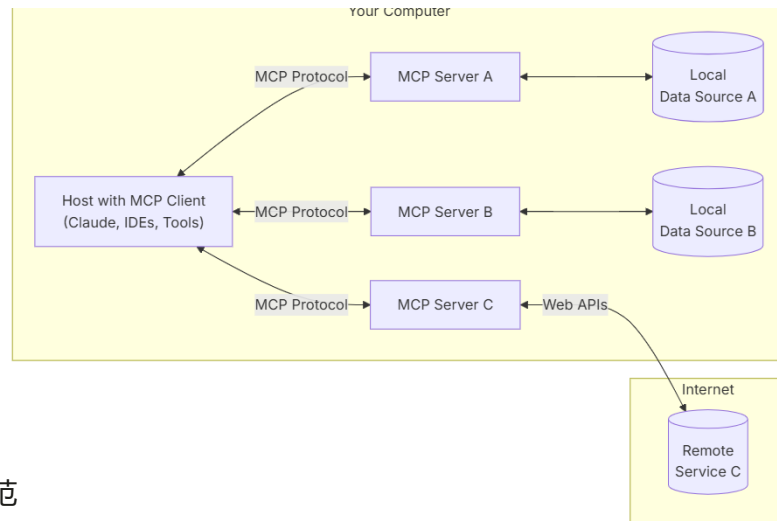
应对M个模型 x N个应用，MCP提高不同应用和资源的兼容和交互

- MCP架构能够通过将设备层的通信与协作责任卸载到MCP服务器, 帮助LLM应用专注于更高层次的任务, 如网络资源调度、故障诊断、网络配置优化等。
- 不仅是用于应用和LLM之间的交互; 通过资源声明、提示模板、工具接口和动态采样等机制, MCP帮助开发者更高效地整合数据库、API、工具等资源, 为LLM应用提供灵活、模块化的架构。
- MCP的设计为LLM应用提供了跨厂商、多客户端的支持。开发者通过MCP实现跨平台兼容, 减少开发成本。
- 在模型知道怎么正确做的基础上, 增加了正确完成操作的概率

1. 标准化协议, 提高不同模型, 尤其是不同应用的兼容性
2. LLM生态的核心组件, 占领开发人员心智的底层框架
3. 协议会影响模型间的交互方式, 交互流量和交互强度等

协议定义接口, 将对大模型agent, 2B业务有重大影响; 强烈建议我司跟踪, MCP也会影响算力的设计

MCP框架

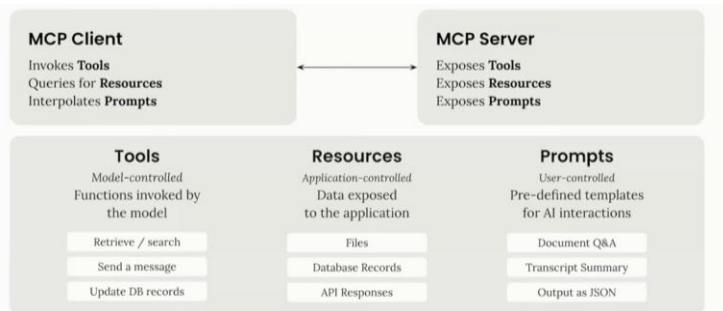


- 标准化规范
 - > 通讯架构标准: 客户端 - 服务器模式、扩展机制、错误处理标准等;
 - > 工具定义标准: 工具定义规范、版本标识方法、URI格式等;
 - > 通信协议标准: 建立连接流程、消息交换格式 (JSON-RPC)、消息类型等;
- 核心概念
 - > 资源 (Resources): 服务端向客户端暴露的数据和内容;
 - > 工具 (Tools): 服务端向客户端提供的可执行的外部函数;
 - > 提示词 (Prompts): 服务端向客户端提供的可重用的提示词模板;
 - > 采样 (Sampling): 客户端向服务端提供的与大模型交互功能;
- 客户端-服务器架构
 - > 主机 (Hosts): 对应Claude Desktop等发起连接的AI应用, 可以管理多个客户端实例;
 - > 客户端 (Clients): 在主机应用程序内部与服务端 1:1 连接, 建立双向通讯;
 - > 服务端 (Servers): 向客户端提供相关资源和工具以扩展 AI 大模型能力;

为什么需要MCP

设计理念

- AI 应用、生态系统和 Agent 的未来发展方向会倾向于 Statefulness，在设计 MCP 协议时，使其stateful
- 这是个 MxN 的问题，也就是多个应用程序与多种集成的难题，而用一种协议解决再合适不过
- MCP 服务器填补了模型在当时自身能力上的空白。模型训练、准备和研究需要耗费大量时间，才能逐步提升其能力。
- 实现了与具体模型无关的构建
- 甚至利用 MCP 服务器构建有 DAG（Directed Acyclic Graph）来实现复杂的交互流程。智能的 MCP 服务器甚至可以利用整个 MCP 服务器生态系统的能力



特性

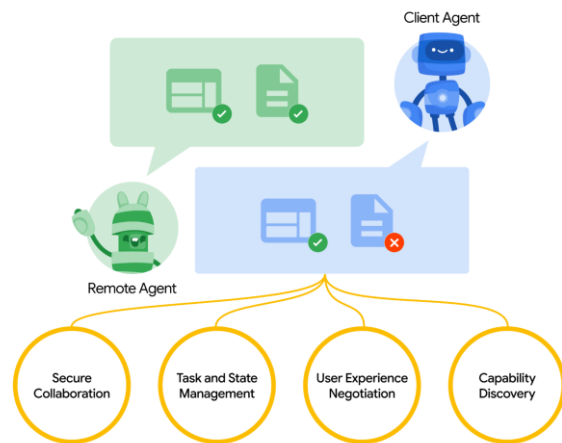
- 在 MCP 的范式下，客户端应用应该拥有完全的控制权。在协议中添加功能，允许服务器开发者对提示、资源和工具这些基本元素进行逻辑分组。这些分组可以被视为不同的 MCP 服务器，然后由用户根据自己的需求将它们组合起来使用。
- MCP 的一个优点在于，构建一些简单的功能非常容易，大约半小时就能搭建好，虽然可能不完美，但足以满足基本需求。最好的入门方法是：选择你喜欢的编程语言，如果有相应的 SDK 就直接使用；构建一个你希望模型能与之交互的工具；搭建 MCP 服务器；将这个工具添加到服务器中；简单地编写一下工具的描述；通过标准输入输出协议将其连接到你喜欢的应用程序；然后观察模型能够如何使用它
- **倾向于返回大量的数据，并相信 LLM 能够从中筛选并提取它所关心的信息**
- **对于模型下一波能力的提升，最大的瓶颈可能在于与外部世界交互的能力**，比如读取外部数据源、采取 Statefulness 的行动。在 Anthropic 工作时，我们非常重视安全的交互，并采取了相应的控制和校准措施。随着 AI 的发展，人们会期望模型具备这些能力，而将模型与外部连接是提升 AI 生产力的关键。MCP 也正是我们对未来发展方向及其重要性的一种押注。
- 任何带有「格式化」（formatted）字样的 API 属性都应该被移除。应该从所有接口获取原始数据。模型肯定足够智能，能够自己对地址等信息进行格式化。所以这部分应该由终端用户来决定

挑战

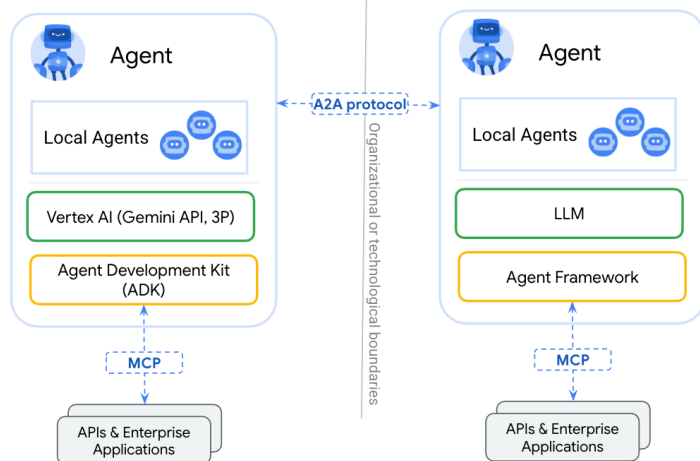
- **【MCP错误调用工具的问题】**一方面取决于你使用的模型，另一方面取决于工具的命名和描述是否足够清晰，能够让模型准确理解，避免混淆。理想的状态是将所有信息提供给 LLM，完全由它来处理一切，这也是 MCP 所设想的未来蓝图。但在现实应用中，客户端应用程序（即 AI 应用）可能需要做一些补充工作，比如筛选工具集，或者利用一个小型且快速的 LLM 先筛选出最相关的工具，然后再传递给大型模型。此外，也可以通过将一些 MCP 服务器设置为其他 MCP 服务器的代理来进行筛选。
- 如果要求 MCP 服务器保持长期持续连接，部署和运营的难度会非常大

A2A协议：定义不同平台不同agent之间的通信协议

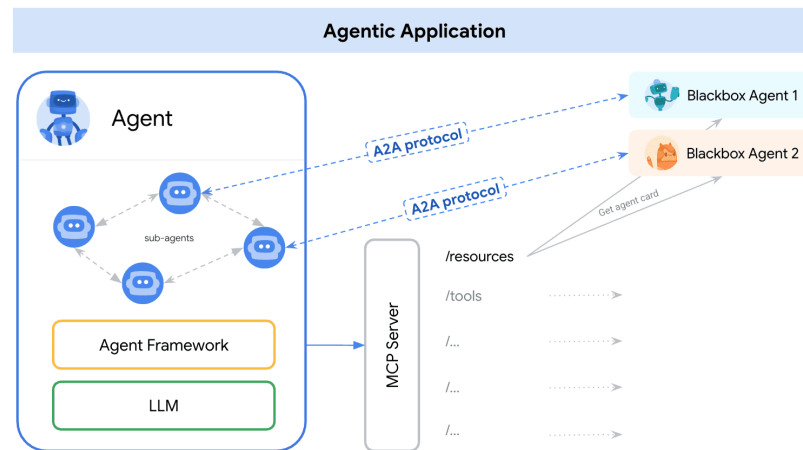
A2A：agent之间任务和管理包括安全管理



A2A与MCP的关系



A2A：和远端agent的通信，获得其信息和状态



A2A：连接不同垂直领域的agent，扩大gemini影响力



启示

- 鸿蒙系统里有不同的app应用，未来会有其agent形态，需要在鸿蒙体系内的agent之间的通信协议
- 模型基于协议的特点微调，使模型天生善用协议功能
- 结合模型，协议层，软件层和硬件层都打通，可以最大化agent的能力，做强平台化能力
- 协议成为行业标准有利于应用厂商调用盘古大模型，拉动算力底座需求